

Collaborative zkSNARKs with Sublinear Prover Time and Constant Proof Size

Zhiyong Fang¹, Sanjam Garg², Bhaskar Roberts², Wenxuan Wu¹, Yupeng Zhang^{3*}

¹Texas A&M University ²University of California, Berkeley

³University of Illinois Urbana-Champaign

Abstract

Collaborative zkSNARKs, proposed by Ozdemir and Boneh in 2022, allow a prover to delegate the generation of zkSNARK proofs to multiple servers, without compromising the confidentiality of the secret witness. They enable the use of zkSNARK techniques on computationally limited devices in critical applications such as blockchains. However, the running time of each server is at least as slow as computing the proof on a single server. Garg et al. attempted to improve the efficiency in their scheme named zkSaaS using packed secret sharing, but the scheme still requires a powerful central server with linear computation, communication and memory usage.

In this paper, we propose a new collaborative zkSNARK scheme with $O(\frac{C}{n} \log \frac{C}{n})$ prover time and $O(1)$ proof size with n servers for a circuit of size C . An adversary compromising less than $\frac{n}{4}$ servers cannot learn any information about the witness. The core of our technique lies in a new zkSNARK scheme for the Plonkish constraint system that is friendly to packed secret sharing. We utilize bivariate polynomials to avoid a large Fast Fourier Transform on the entire witness, which was the major bottleneck in prior work. We also construct permutation constraints based on logarithmic derivatives and univariate sumcheck to avoid the computation of prefix products. Finally, we build a bivariate polynomial commitment scheme that can be computed directly on packed secret shares. Experimental results show that for a circuit of size 2^{20} , with 128 servers, our scheme can accelerate the proof generation by $36.2\times$ compared to running the zkSNARK on a single server. The prover time of our system is $25.9\times$ faster than the prior work of zkSaaS. The proof size of our scheme is only 960 Bytes.

1 Introduction

Zero-Knowledge Succinct Non-interactive Argument of Knowledge (zkSNARK) schemes have seen prominent developments in recent years. They have been widely adopted in

blockchain applications to build privacy-preserving cryptocurrencies [5], zkRollups, zkEVMs [1] and zkBridges [47]. It has also found other applications, such as zero-knowledge machine learning [22, 32, 42], zero-knowledge middle boxes [25] and photo authenticity [12, 34]. A zkSNARK allows a prover to convince a verifier that a statement on the prover's witness is true, without revealing the secret witness to the verifier, ensuring both confidentiality and integrity simultaneously. However, it usually introduces a high computational overhead for the prover to generate the proof. Therefore, to enable the use of zkSNARKs on computationally limited devices, such as regular clients in the blockchain applications, it is desirable to delegate the proof generation to powerful cloud servers. Meanwhile, as these servers are not trusted, they should not learn the secret witness to protect the confidentiality.

Ozdemir and Boneh initiated the study of this setting in [35] named *Collaborative zkSNARKs* and proposed a solution utilizing secure multiparty computations (MPC). In this solution, a client can secret-share the witness to multiple servers, who execute an MPC protocol to generate the proof of several popular zkSNARK schemes such as Groth16 [24], Plonk [17] and Marlin [10] with constant proof size. The servers do not learn any information about the witness as long as they do not collude. However, the major limitation of the scheme is that the running time of each server is as slow as running the original zkSNARK protocol on the witness in plaintext. That is, there is no efficiency gain in Collaborative zkSNARKs, even though multiple servers are used. To address this issue, Garg et. al [20] proposed zkSaaS (zkSNARKs-as-a-service), which uses packed secret sharing (PSS) to make the work on each server sublinear in the complexity of generating the original zkSNARK proof. Unfortunately, the paper does not fully solve the problem and there still has to be one central server with linear running time and memory usage due to the challenging computations of Fast Fourier Transform (FFT) and prefix multiplications that commonly appear in the zkSNARKs of Groth16 and Plonk.

More recently, Liu et. al [33] proposed a collaborative zkSNARK protocol with sublinear prover time on all servers.

*Authors are listed in alphabetical order.

They switch to zkSNARKs based on multivariate polynomials [9, 46] to avoid FFT. However, the scheme incurs a logarithmic number of rounds between the servers and has a logarithmic proof size and verifier time. The larger proof size and verifier time are not desirable in applications such as blockchains as it is costly to post the proof on-chain and verify it in smart contracts. The logarithmic number of interactions also incur overhead due to round-trip-time and synchronization between the servers.

1.1 Our Contributions

In this paper, we solve the original open problem proposed in [20] by developing a collaborative zkSNARK scheme with constant proof size and sublinear prover time on each server. In particular, our contributions include:

- First, we introduce a new variant of the polynomial interactive oracle proof (PIOP) in Plonk that is compatible with packed secret sharing. Instead of avoiding FFT as in [33], we break the large FFT in Plonk into multiple small FFTs via a bivariate polynomial constraint system, inspired by Pianist [31]. This constraint system is single-instruction-multiple-data (SIMD), and thus can be computed efficiently in MPC with PSS. Moreover, to avoid the prefix product computation in Plonk and Pianist, we replace its permutation constraint by the lookup argument in [26]. The computation is additive after an inversion, and thus can be computed in one round in MPC. The same technique has been used in DFS [27] with a multivariate PIOP and a logarithmic proof size. However, unlike [27], we utilize a generalization of the univariate sumcheck technique to prove the lookup argument, significantly simplifying the protocol while obtaining a constant proof size. Finally, we show how to make the protocol zero-knowledge, which was not achieved in [31]. Putting them together, our new PIOP can be computed by an MPC protocol with $O(\frac{C}{n} \log \frac{C}{n})$ computation on each server in $O(1)$ round, where C is the size of the zkSNARK statement modeled as a circuit, and n is the number of servers.
- Second, we compile our PIOP into a zkSNARK with the polynomial commitment of bivariate KZG [29, 31]. This is particularly challenging, as the KZG polynomial commitment is not compatible with PSS, and the prior work of zkSaaS [20] has to implement the MSM in MPC with additional communications and computations. In this paper, we show that after preprocessing, it is possible to make the prover time of the PCS on each server $O(\frac{C}{n})$ with minimal communication between each other. The computation on each server is exactly the same as that in KZG on plaintext, and existing optimizations such as the Pippenger’s algorithm [38] can be applied to improve the efficiency. Our scheme can be made maliciously secure via the compiler

Scheme	Prover Time	# of Rounds	Proof Size
[11, 35, 48]	$O(C \log C)$	$O(1)$	$O(1)$
zkSaaS [20]	Central: $O(C \log C)$ Others: $O(\frac{C}{n} \log \frac{C}{n})$	$O(1)$	$O(1)$
[33]	$O(\frac{C}{n})$	$O(\log C)$	$O(\log C)$
DFS [27]	$O(\frac{C}{n})^*$	$O(\log C)$	$O(\log C)$
Ours	$O(\frac{C}{n} \log \frac{C}{n})$	$O(1)$	$O(1)$

Table 1: Comparisons between different collaborative zk-SNARKs schemes. C is the number of constraints, n is the number of servers. *DFS achieves the speedup via parallelization with multiple nodes on each server.

in [21]. Table 1 shows the comparison between our scheme and existing Collaborative zkSNARKs schemes.

- Finally, we have fully implemented our collaborative zk-SNARK protocol. Experimental results show that with 128 servers, our protocol can generate a zkSNARK proof for 2^{20} constraints in 13.5 seconds. This is $36.2\times$ faster than running the original Plonk protocol on a single machine. Our protocol is $25.9\times$ faster than zkSaaS [20], and is even $4.7\times$ faster than the scheme in [33], which has a linear prover time and logarithmic proof size.

1.2 Related Work

Collaborative zkSNARKs. In addition to the papers mentioned above, earlier works [13, 28, 40] developed MPC protocols for the zkSNARKs of Pinocchio [37], Marlin [10] and Ligerio [2]. In [11], Chiesa et al. considered a different setting where the client remains online in the MPC protocol, and proposed an efficient collaborative ZK protocol named EOS with the help of the client. Siniel [48] further removes the online client from EOS and achieves malicious security efficiently. In [45], Wu et al. proposed a Collaborative zkSNARKs protocol for the zkSNARK based on the sumcheck protocol and the GKR protocol, and with a special focus on machine learning computations such as matrix multiplications. In all of these prior work, the running time of each server is asymptotically the same as zkSNARK prover in the non-delegation setting, and is usually even slower in practice due to the overhead of the MPC protocol.

Very recently, Hu et al. proposed a scheme named DFS in [27] that reduces the communication complexity to sub-linear in the size of the circuit. It relies on a new zkSNARK scheme based on multivariate polynomials that is delegation friendly. The scheme also achieves sublinear prover time via parallel computation with multiple machines on each server,

and thus the setting and the security guarantees are different from our work with packed secret sharing. The proof size and round complexity is $O(\log C)$ due to the use of multivariate polynomials and sumcheck protocols. Constructing a collaborative zkSNARKs protocol with sublinear prover time, sublinear communication and constant proof size remains an interesting open problem.

Distributed zkSNARKs. A different setting of distributed generation of zkSNARK proofs, or public delegations, has been considered in [30, 31, 39, 43, 44, 47]. These schemes accelerate the proof generation of zkSNARKs via distributed computing using multiple machines, but do not protect the confidentiality of the witness from these machines.

Single server delegation with homomorphic encryptions. Finally, there is a recent line of work delegating the zkSNARK computation to a single server using homomorphic encryptions (HE) [3, 18, 23, 49]. They do not require multiple non-colluding servers as in collaborative zkSNARKs. However, due to the heavy machinery of HE, they incur high overheads in practice. For example, as shown in [3], it takes 207 seconds to generate the proof of the PCS part of zkSNARK for a polynomial of size 2^{15} , which is orders of magnitude slower than collaborative zkSNARKs.

2 Preliminaries

We use $[N]$ to denote the set $\{0, 1, 2, \dots, N-1\}$.

Polynomials. For any univariate polynomial $f(X)$ of degree $T-1$, we denote f by its evaluations over the domain $\Omega_x = \{\omega_x^0, \dots, \omega_x^{T-1}\}$ generated by the T -th root of unity ω_x such that $\omega_x^T = 1$. For simplicity we assume that T is a power of 2. As a result, $f(X)$ is represented as a vector $(f(\omega_x^j))_{j \in [T]}$.

For any bivariate polynomial $g(Y, X)$ of degree $\ell-1$ in Y , and degree $T-1$ in X . We represent $g(Y, X)$ by its evaluations $g(\omega_y^i, \omega_x^j)$ for $i \in [\ell], j \in [T]$, where ω_y is the ℓ -th root of unity.

With these representations, to evaluate $f(X)$ at a point α , we write $f(x) = \sum_{j \in [T]} L_T^{(j)}(\alpha) \cdot f(\omega_x^j)$, where $L_T^{(j)}$ is the j -th Lagrange basis polynomial defined by Ω_x . Similarly, $g(\beta, \alpha) = \sum_{i \in [\ell], j \in [T]} L_\ell^{(i)}(\beta) \cdot L_T^{(j)}(\alpha) \cdot g(\omega_y^i, \omega_x^j)$, where $L_\ell^{(i)}$ is the i -th Lagrange basis polynomial defined by Ω_y .

Bilinear Pairings. Let $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_T$ be cyclic groups of prime order p , and let $g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ be their generators. A bilinear pairing is an efficiently computable map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ satisfying the following properties:

- **Bilinearity:** For all $u \in \mathbb{G}_1, v \in \mathbb{G}_2$, and $a, b \in \mathbb{Z}_p$, we have $e(u^a, v^b) = e(u, v)^{ab}$.
- **Non-degeneracy:** $e(g_1, g_2) \neq 1$.

We use $\text{bp} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e) \leftarrow \text{BilGen}(1^\lambda)$ to denote the generation of parameters of a bilinear map.

2.1 Secure Multiparty Computation

Secure multiparty computations enable multiple parties to jointly evaluate a function f over their inputs, while revealing nothing beyond the output. We say an MPC protocol *securely computes* an ideal functionality f against semi-honest adversaries if for any semi-honest adversaries, the adversary's view in the protocol and the outputs of all parties can be simulated in an ideal world in which the simulator (a.k.a. the ideal-world adversary) only interacts with the ideal functionality.

Secret Sharing. We will use MPC protocols based on Shamir's secret sharing [41] and packed secret sharing [16]. In an (n, t) Shamir secret sharing scheme, a secret $x \in \mathbb{F}$ is embedded in the constant term of a random polynomial of degree t , and the shares are the evaluations of the polynomial at n points. An adversary corrupting no more than t parties cannot learn anything about the secret, while $t+1$ parties can reconstruct the secret. We use $[x]$ to denote the Shamir sharing, and $[x]_i$ to denote the share of party i . $[x] \leftarrow \text{share}(\mathbb{F}, x, t)$ computes the shares from the secret x , and $x \leftarrow \text{open}(\mathbb{F}, [x], t)$ reconstructs the secret from the shares. In an (n, t, ℓ) packed secret sharing scheme, we can represent a vector of ℓ secrets $\mathbf{x} \in \mathbb{F}^\ell$ as ℓ evaluation points of a random polynomial of degree $D = t + \ell$. Then we can share n additional evaluations with n parties. The shares are denoted as $[[\mathbf{x}]]$, and $[[\mathbf{x}]]_i$ denotes the share of the i -th party. $[[\mathbf{x}]] \leftarrow \text{pshare}(\mathbb{F}, \mathbf{x}, D)$ computes the sharing and $\mathbf{x} \leftarrow \text{popen}(\mathbb{F}, [[\mathbf{x}]], D)$ computes the reconstruction. Additionally, let $\langle\langle \mathbf{x} \rangle\rangle$ be a sharing of $\mathbf{x}$ with a polynomial of degree $n-1$.

Due to the linearity of the sharing schemes, additions and scalar multiplications can be performed on shares without any interaction between the parties. In packed secret sharing, additions and multiplications can be performed element-wise between two secret vectors, as in SIMD computations. By setting $\ell = O(n)$ and $t = O(n)$, the amortized computation and communication complexity per secret can be reduced to $O(1)$ per party.

Sub-protocols. We will use several standard sub-protocols for multiplication, permutation, and randomness generation, etc. The protocols are given in Appendix A and fig. 4, and they securely compute the ideal functionalities listed below against a semi-honest adversary. Most of these functionalities come from [19]. Each of the following functionalities receives from the adversary a list of the corrupted parties and the corrupted parties' shares of the output, and they send to the honest parties their shares of the output.

- $f_{\text{rand}}(n, t, l)$: Samples $\mathbf{r} \leftarrow \mathbb{F}^l$ and generates a random degree- $(t+l)$ packed sharing of $\mathbf{r}$ such that the corrupted parties' shares of $[[\mathbf{r}]]$ match the ones supplied by the adversary.
- $f_{\text{double-rand}}(n, t, l)$: Samples $\mathbf{r} \leftarrow \mathbb{F}^l$ and generates two packed sharings of $\mathbf{r}$; one is a random degree- $(t+l)$ sharing,

and the other is a random degree- $(n-1)$ sharing, and the shares of the corrupted parties match the ones supplied by the adversary.

- $f_{\text{pack-mult}}(n, t, l)$: Takes as input the shares of $[[\mathbf{a}]]$, $[[\mathbf{b}]]$ from all parties. Then the functionality computes

$$\begin{aligned}\mathbf{a} &= \text{popen}(\mathbb{F}, [[\mathbf{a}]], t+l) \\ \mathbf{b} &= \text{popen}(\mathbb{F}, [[\mathbf{b}]], t+l) \\ \mathbf{c} &= \mathbf{a} \odot \mathbf{b}\end{aligned}$$

where $\odot$ is component-wise multiplication. Finally, the functionality generates a random packed sharing of $\mathbf{c}$ such that the corrupted parties' shares of $[[\mathbf{c}]]$ match the ones supplied by the adversary.

- $f_{\text{permute}}(p_{\text{perm}}, n, t, l)$: Takes as input the shares of $[[\mathbf{x}]]$ from all parties. Then the functionality computes

$$\begin{aligned}\mathbf{x} &= \text{popen}(\mathbb{F}, [[\mathbf{x}]], t+l) \\ \mathbf{y} &= p_{\text{perm}}(\mathbf{x})\end{aligned}$$

where p_{perm} permutes the l entries of $\mathbf{x}$. Finally, the functionality generates a random packed sharing of $\mathbf{y}$ such that the corrupted parties' shares of $[[\mathbf{y}]]$ match the ones supplied by the adversary.

- $f_{\text{ps2rs}}(n, t, l)$: Takes as input the shares of $[[\mathbf{x}]]$ from all parties. Then the functionality computes $\mathbf{x} = \text{popen}(\mathbb{F}, [[\mathbf{x}]], t+l)$. Finally, for each $k \in [l]$, the functionality generates a random Shamir sharing of x_k such that the corrupted parties' shares of $[x_k]$ match the ones supplied by the adversary.
- $f_{\text{invert}}(n, t, l)$: Converts a sharing of $(x_1, \dots, x_l)$ to a sharing of $(x_1^{-1}, \dots, x_l^{-1})$, except that it may abort with negligible probability.

Formally, the functionality takes as input the shares of $[[\mathbf{x}]]$ from all parties. Then the functionality computes:

$$\mathbf{x} = \text{popen}(\mathbb{F}, [[\mathbf{x}]], t+l)$$

Next, the functionality samples $\mathbf{r} \leftarrow \mathbb{F}^l$. If $x_k = 0$ or $r_k = 0$ for any $k \in [l]$, then the functionality sends $\perp$ to all parties, sends to the adversary the indices k for which $x_k = 0$ or $r_k = 0$, and finally halts. Otherwise, the functionality continues and computes:

$$\mathbf{y} = (x_1^{-1}, \dots, x_l^{-1})$$

Finally, the functionality generates a random packed sharing of $\mathbf{y}$ such that the corrupted parties' shares of $[[\mathbf{y}]]$ match the ones supplied by the adversary.

The following functionality receives from the adversary a list of the corrupted parties and outputs a plaintext value rather than a secret-shared value.

- $f_{\text{eval-open}}(n, t, l, s_x, s_y)$ takes a secret-shared polynomial $[[f(X, Y)]]$ and computes $g_1^{f(s_x, s_y)}$. Formally, the functionality takes as input $[[f(X, Y)]] := ([[f_0]], \dots, [[f_{T-1}]])$, where for each $i \in [T]$, $\mathbf{f}_i = (f(\omega_x^i, \omega_y^0), \dots, f(\omega_x^i, \omega_y^{l-1}))$. Then the functionality reconstructs $f(\omega_x^i, \omega_y^j)$ for each $(i, j) \in [T] \times [M]$. Then the functionality computes:

$$f(s_x, s_y) = \sum_{i \in [T], j \in [l]} L_T^{(i)}(s_x) \cdot L_\ell^{(j)}(s_y) \cdot f(\omega_x^i, \omega_y^j)$$

Finally, the functionality outputs $g_1^{f(s_x, s_y)}$ to all parties.

2.2 Collaborative zkSNARKs

A collaborative zkSNARK [35] is a multiparty computation that generates a zkSNARK as its output. It satisfies completeness, knowledge soundness, succinctness, and t -zero-knowledge, defined in definition 2.1. *Completeness* says that if the witness is valid and the MPC is performed honestly, then the resulting proof will pass verification with overwhelming probability. *Succinctness* ensures that the verifier's work is less than the time required to verify the original witness. *Knowledge soundness* says that if the provers can convince the verifier to accept with some probability, then there is an extractor that can extract from the provers a valid witness w , with similar probability. Finally, *t -zero-knowledge* ensures that no subset of t parties or fewer will learn anything about the witnesses of the other parties, except whether the combined witness is valid. We define these terms formally below.

Variables: Let $\mathfrak{R}$ be the relation. Let $(x, w) \in \mathfrak{R}$ be a valid instance-witness pair. Let $V_{\mathfrak{R}}$ be a polynomial-sized circuit that verifies (x, w) , and let $|V_{\mathfrak{R}}|$ be the size of $V_{\mathfrak{R}}$'s circuit. Next, let t be the maximum number of malicious provers. Let λ be the security parameter. Let H be a random oracle, sampled from the appropriate domain $\mathcal{U}(\lambda)$, and let $H[\mu]$ be the function H reprogrammed at some points specified by μ .

Definition 2.1. [21, 35] A collaborative zkSNARK is a tuple of algorithms $(\text{cZK.Setup}^H, \text{cZK.Prove}^H, \text{cZK.Verify}^H)$ that satisfies the following properties:

Completeness: For any $(x, w) \in \mathfrak{R}$:

$$\Pr \left[\text{cZK.Verify}^H(\text{pp}, x, \pi) = 0 : \begin{array}{l} \text{pp} \leftarrow \text{cZK.Setup}^H(1^\lambda, \mathfrak{R}) \\ \pi \leftarrow \text{cZK.Prove}^H(\text{pp}, x, w) \end{array} \right] = \text{negl}(\lambda)$$

Knowledge Soundness: For any x and any efficient provers $\vec{P}$, there exists an efficient extractor Ext with the following properties: Ext gets oracle access to H . Ext can run the provers multiple times, reprogramming H each time, and Ext receives only the provers' final output from each run. This is denoted as $\text{Ext}^{H, \vec{P}^H}$. Then

$$\Pr \left[(x, w) \in \mathfrak{R} : \begin{array}{l} H \xleftarrow{\$} \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{cZK.Setup}^H(1^\lambda, \mathfrak{R}) \\ w \leftarrow \text{Ext}^{H, \vec{P}^H}(\text{pp}, x) \end{array} \right]$$

is greater than or equal to

$$\Pr \left[\text{cZK.Verify}^H(\text{pp}, x, \pi) = 1 : \begin{array}{l} H \xleftarrow{\$} \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{cZK.Setup}^H(1^\lambda, \mathfrak{R}) \\ \pi \leftarrow \tilde{P}^H(\text{pp}, x) \end{array} \right] - \text{negl}(\lambda)$$

Succinctness: The size of π and the runtime of $\text{cZK.Verify}^H(\text{pp}, x, \pi)$ are $o(|V_{\mathfrak{R}}|)$.

t -Zero-Knowledge: For any PPT malicious adversary $\mathcal{A}$ controlling $k \leq t$ provers $(P_{i_1}, \dots, P_{i_k})$, there exists an efficient simulator Sim such that for all (x, w) , and for all efficient distinguishers D , the difference between

$$\Pr \left[D^{H[\mu]}(\text{tr}) = 1 : \begin{array}{l} H \xleftarrow{\$} \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{cZK.Setup}^H(1^\lambda, \mathfrak{R}) \\ b \leftarrow V_{\mathfrak{R}}(x, w) \in \{0, 1\} \\ (\text{tr}, \mu) \leftarrow \text{Sim}^H(\text{pp}, x, w_{i_1}, \dots, w_{i_k}, b) \end{array} \right]$$

and

$$\Pr \left[D^H(\text{tr}) = 1 : \begin{array}{l} H \xleftarrow{\$} \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{cZK.Setup}^H(1^\lambda, \mathfrak{R}) \\ \text{tr} \leftarrow \text{View}_{\mathcal{A}}^H[x, w] \end{array} \right]$$

is negligible. Here, tr is the protocol transcript, and $\text{View}_{\mathcal{A}}^H[x, w]$ is $\mathcal{A}$'s view of the protocol.

3 Our Polynomial IOP for Plonkish Constraints

In this section, we present our polynomial IOP for Plonkish constraints that is compatible with PSS. Our starting point is the protocol with bivariate polynomials proposed in [31].

Polynomial encoding. A circuit C is divided into ℓ sub-circuits of size $T = \frac{|C|}{\ell}$ each. For each sub-circuit C_i , we define degree T wire polynomials $a_i(X), b_i(X), o_i(X)$ such that $a_i(\omega_x^j), b_i(\omega_x^j)$, and $o_i(\omega_x^j)$ equal the left input, the right input, and the output of gate $j \in [T]$ respectively, where ω_x is the T -th primitive root of unity.

Gate constraint. We also define degree- T gate polynomials $q_{\text{add},i}(X), q_{\text{mul},i}(X), q_{\text{cst},i}(X)$ based on the types of gates in C_i . $q_{\text{add},i}(\omega_x^j) = 1$ ($q_{\text{mul},i}(\omega_x^j) = 1$) if the j -th gate of C_i is an addition (multiplication) gate, and is 0 otherwise. $q_{\text{cst},i}(\omega_x^j)$ equals to the public constant value if the j -th gate is a constant gate, and is 0 otherwise. With these polynomials, we have:

$$g_i(X) := -o_i(X) + q_{\text{add},i}(X)(a_i(X) + b_i(X)) + q_{\text{mul},i}(X)a_i(X)b_i(X) + q_{\text{cst},i}(X) \quad (1)$$

equals to 0 for all $X \in \Omega_x = \{\omega_x^0, \dots, \omega_x^{T-1}\}$.

To combine the gate constraints of all ℓ subcircuits, we define the bivariate polynomials: for $s \in \{a, b, o, q_{\text{add}}, q_{\text{mul}}, q_{\text{cst}}\}$, $S(Y, X) = \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y) s_i(X)$, where $L_\ell^{(i)}(Y)$ is the i -th Lagrange polynomial defined by the ℓ -th primitive root of unity ω_Y . The bivariate gate constraint is:

$$G(Y, X) := -O(X) + Q_{\text{add}}(Y, X)(A(Y, X) + B(Y, X)) + Q_{\text{mul}}(Y, X)A(Y, X)B(Y, X) + Q_{\text{cst}}(Y, X) \quad (2)$$

equals to 0 for all $X \in \Omega_y, Y \in \Omega_y$.

Permutation constraint. In addition to the gate constraint, the verifier needs to check that the connections of wires are correct as defined by the circuit. Note that we do support general circuits with connections between different sub-circuits. For example, if the output of the j_1 -th gate in the i_1 -th sub-circuit is used as the left input of the j_2 -th gate in the i_2 -th sub-circuit, we must have $o_{i_1}(\omega_x^{j_1}) = a_{i_2}(\omega_x^{j_2})$.

To enforce the correct connections, it suffices to check that the ordered list of the left inputs, right inputs and outputs $(\mathbf{a}, \mathbf{b}, \mathbf{o})$ remains unchanged after applying a permutation σ defined by the circuit connections, which was proposed in Plonk [17]. This permutation constraint is checked by a permutation argument, which was extended to the bivariate case in Pianist. In the permutation argument, a polynomial z defined by a prefix product is computed and committed by the prover: $z(\omega^j) := \prod_{k=0}^{j-1} \frac{f(\omega^k)}{f'(\omega^k)}$.

We omit their formal definitions and refer the readers to [17, 31] for the equations. Unfortunately, this prefix product introduces a significant challenge for Collaborative zk-SNARKs. As f and f' are secret-shared between the servers, the multiplication depth is linear in the size of the circuit when implemented naively, which would lead to a linear number of rounds in MPC. zkSaaS proposed an approach to compute it in a constant number of rounds, but it requires a central server with a linear computation and communication complexity in the size of the circuit.

3.1 Permutation Argument via Logarithmic Derivatives

To address this issue, we take a different approach to prove the permutation constraint using the technique of logarithmic derivatives proposed in [14]. To prove that two vectors $\mathbf{u}, \mathbf{v}$ are permutations of each other, it suffices to show that for a random γ ,

$$\sum_{i=0}^{n-1} \frac{1}{\gamma + u_i} = \sum_{i=0}^{n-1} \frac{1}{\gamma + v_i}.$$

Adapting the technique to our permutation constraint, we define the permutation polynomials $(\sigma_{Y,s,i}(X), \sigma_{X,s,i}(X))$ for $s \in \{a, b, o\}$ such that if the wire encoded in polynomial s of the j -th gate in the i -th sub-circuit is connected to the wire encoded in polynomial s' of the j' -th gate in the i' -th sub-circuit, then $\sigma_{Y,s,i}(\omega_X^j) = \omega_Y^{j'}$ and $\sigma_{X,s,i}(\omega_X^j) = k_{s'} \omega_X^{j'}$. That is, $\sigma_{Y,s,i}(X)$ encodes the index of the sub-circuit each wire connects to, $\sigma_{X,s,i}(X)$ encodes the index within that sub-circuit. $k_a = 1$, k_b is any quadratic non-residue, and k_o is a quadratic non-residue not contained in $k_b \Omega_X$. They are used to separate the three wires encoded in polynomials a, b, o . With these polynomials, the correct connection of the circuit is equivalent to the condition that the triples $(s(X), \sigma_{X,s,i}(X), \sigma_{Y,s,i}(X))$ for $s \in \{a, b, o\}$, all $i \in [\ell]$ and all $X \in \Omega_X$, is a permutation of

$(s(X), k_s X, \omega_y^i)$. To show this permutation using logarithmic derivatives, given random challenges η_X, η_Y, γ , we define:

$$\begin{aligned} ta_i(X) &= (a_i(X) + \eta_X \sigma_{X,a,i}(X) + \eta_Y \sigma_{Y,a,i}(X) + \gamma) \\ tb_i(X) &= (b_i(X) + \eta_X \sigma_{X,b,i}(X) + \eta_Y \sigma_{Y,b,i}(X) + \gamma) \\ to_i(X) &= (o_i(X) + \eta_X \sigma_{X,o,i}(X) + \eta_Y \sigma_{Y,o,i}(X) + \gamma) \end{aligned}$$

$$\begin{aligned} ta'_i(X) &= (a_i(X) + \eta_X k_a X + \eta_Y \omega_y^i + \gamma) \\ tb'_i(X) &= (b_i(X) + \eta_X k_b X + \eta_Y \omega_y^i + \gamma) \\ to'_i(X) &= (o_i(X) + \eta_X k_o X + \eta_Y \omega_y^i + \gamma). \end{aligned}$$

Let f and f' be

$$\begin{aligned} f_i(X) &:= \frac{1}{ta(X)} + \frac{1}{tb(X)} + \frac{1}{to(X)} \\ f'_i(X) &:= \frac{1}{ta'(X)} + \frac{1}{tb'(X)} + \frac{1}{to'(X)}, \end{aligned}$$

the permutation is implied by:

$$\sum_{i \in [\ell]} \sum_{X \in \Omega_x} f_i(X) = \sum_{i \in [\ell]} \sum_{X \in \Omega_x} f'_i(X)$$

This equation is the derivative of the polynomial constraint with prefix products in Pianist [31, Equation 13]. It only involves additions, thus is very friendly to Collaborative zk-SNARKs.

To verify the equation above, we define $z_i(\omega_x^j) = f_i(\omega_x^j)$ and $z'_i(\omega_x^j) = f'_i(\omega_x^j)$ for $j \in [T]$. To check their correctness, we have

1. $p_{i,0}(X) := ta_i(X) \cdot tb_i(X) \cdot to_i(X) \cdot (z_i(X) - f_i(X))$
2. $p_{i,1}(X) := ta'_i(X) \cdot tb'_i(X) \cdot to'_i(X) \cdot (z'_i(X) - f'_i(X))$

equal to 0 at all $X \in \Omega_x$. Similar to the gate constraint, we can combine the polynomials of all ℓ subcircuits by defining the bivariate polynomials: for $s \in z, z', ta, tb, to, ta', tb', to'$, $S(Y, X) = \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y) s_i(X)$, where $L_\ell^{(i)}(Y)$ is the i -th Lagrange polynomial defined by the ℓ -th root of unity ω_Y . Then we have the following bivariate polynomial equations that are 0 for all $X \in \Omega_x, Y \in \Omega_y$:

$$\begin{aligned} P_0(Y, X) &= TA(Y, X)TB(Y, X)TO(Y, X)Z(Y, X) - TA(Y, X) \\ &\quad TB(Y, X) - TA(Y, X)TO(Y, X) - TB(Y, X)TO(Y, X) \\ P_1(Y, X) &= TA'(Y, X)TB'(Y, X)TO'(Y, X)Z'(Y, X) - TA'(Y, X) \\ &\quad TB'(Y, X) - TA'(Y, X)TO'(Y, X) - TB'(Y, X)TO'(Y, X) \end{aligned} \quad (3)$$

It remains to prove that

$$\sum_{i \in [\ell]} \sum_{X \in \Omega_x} (z_i(X) - z'_i(X)) = 0,$$

or equivalently

$$\sum_{Y \in \Omega_y} \sum_{X \in \Omega_x} (Z(Y, X) - Z'(Y, X)) = 0. \quad (4)$$

We rely on the following lemma, which generalizes the univariate sumcheck technique [6] to the bivariate case:

Lemma 1. *Let $f(X, Y) = \sum_{i,j \geq 0} \alpha_{i,j} X^i Y^j$ be a bivariate polynomial such that each term $\alpha_{i,j} X^i Y^j$ satisfies: $0 < i < T$ or $0 < j < \ell$ or $(i, j) = (0, 0)$. Then for any cyclic multiplicative subgroups Ω_x, Ω_y of sizes T, ℓ respectively,*

$$\sum_{(X,Y) \in \Omega_x \times \Omega_y} f(X, Y) = T \ell f(0, 0)$$

Proof. Let ω_x and ω_y be generators for Ω_x and Ω_y respectively.

For any $0 < i < T$, $\omega_x^i \neq 1$, so:

$$\sum_{X \in \Omega_x} X^i = \sum_{t \in \{0, \dots, T-1\}} \omega_x^{t \cdot i} = \frac{1 - \omega_x^{T \cdot i}}{1 - \omega_x^i} = \frac{1 - 1}{1 - \omega_x^i} = 0$$

Likewise, for any $0 < j < \ell$,

$$\sum_{Y \in \Omega_y} Y^j = 0$$

Next, let us sum $f(X, Y)$ over all elements of the subgroups:

$$\begin{aligned} \sum_{X \in \Omega_x, Y \in \Omega_y} f(X, Y) &= \sum_{X \in \Omega_x, Y \in \Omega_y} \sum_{i,j \geq 0} \alpha_{i,j} X^i Y^j \\ &= \sum_{i,j \geq 0} \alpha_{i,j} \cdot \left(\sum_{X \in \Omega_x} X^i \right) \cdot \left(\sum_{Y \in \Omega_y} Y^j \right) \end{aligned}$$

If $(i, j) = (0, 0)$, then $\sum_{X \in \Omega_x} X^i = T$ and $\sum_{Y \in \Omega_y} Y^j = \ell$. For all other (i, j) , either $0 < i < T$ or $0 < j < \ell$. In that case, $\sum_{X \in \Omega_x} X^i = 0$ or $\sum_{Y \in \Omega_y} Y^j = 0$.

To conclude:

$$\sum_{X \in \Omega_x, Y \in \Omega_y} f(X, Y) = \alpha_{0,0} \cdot T \cdot \ell = T \ell f(0, 0)$$

□

Applying the technique to $Z(Y, X) - Z'(Y, X)$, to show that Equation 4 holds, it suffices to show that the constant term is 0. Therefore, the prover computes $Q_X(Y, X), Q_Y(Y, X)$ such that: $Z(Y, X) - Z'(Y, X) = Q_X(Y, X)X + Q_Y(Y, X)Y$. The degree of $Q_X(Y, X)$ in X must be less than $T - 2$, and the degree of $Q_Y(Y, X)$ in Y must be less than $\ell - 2$, which can be efficiently checked using the KZG polynomial commitment via the technique in [10]. Moreover, $Q_X(Y, X) = \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y) q_i(X)$, where $q_i(X) = \frac{(z_i(X) - z'_i(X)) - w_i}{X}$ and $w_i = z_i(0) - z'_i(0)$ is the constant term of $z_i(X) - z'_i(X)$.

3.2 Combining Gate and Permutation Constraints

To prove both the gate and the permutation constraints, we take their random linear combination with the randomness λ from the verifier and checks that

$$\begin{aligned} G(Y, X) + \lambda P_0(Y, X) + \lambda^2 P_1(Y, X) \\ = V_X(X)H_X(Y, X) + V_Y(Y)H_Y(Y, X) \end{aligned} \quad (5)$$

$$\text{and } Z(Y, X) - Z'(Y, X) = Q_X(Y, X)X + Q_Y(Y, X)Y \quad (6)$$

where $V_X(X) = X^T - 1$ is the vanishing polynomial of Ω_X and $V_Y(Y) = Y^\ell - 1$ is the vanishing polynomial of Ω_Y . Here $H_X(Y, X)$ is computed as $H_X(Y, X) = \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y)h_i(X)$, where $h_i(X) = \frac{g_i(X) + \lambda p_{i,0}(X) + \lambda^2 p_{i,1}(X)}{X^T - 1}$ only depends on the i -th subcircuit.

We present our full PIOP protocol in Figure 1. The main difference from the description above is that the prover adds blinding values $r_1, \dots, r_{21}$ to the committed polynomials to achieve zero knowledge. Intuitively, a random polynomial with 3 blinding values is added to each of A, B, O, Z, Z', H_X, Q_X , because the polynomial commitment scheme we use in Section 4.2 reveals 3 evaluations of the polynomial: one evaluation at (α, β) , one in the commitment, and one more in the opening proof. The blinding values ensure that these evaluations do not reveal any information about the witness. Moreover, the random polynomials, such as $(r_1X + r_2Y + r_3)V_X(X)$ in $A(Y, X)$ and $(r_{10}X + r_{11}Y + r_{12})XV_Y(Y)$ in $Z(Y, X)$, are chosen such that Equations 5 and 6 still hold with the randomized polynomials. See Section 4.3 for the reasoning of zero knowledge.

4 Our Collaborative zkSNARK Construction

In this section, we present the construction of our collaborative zkSNARK based on the PIOP in Section 3 and the bivariate KZG polynomial commitment scheme.

Encoding of polynomials. The main advantage of our PIOP in Figure 1 is that the computations in Steps (1)-(3) are completely SIMD on variable X , and thus is fully compatible with PSS. The computation in Step (4) on Y is not SIMD, but the size of the computation only depends on ℓ , and we can switch to regular shares without hurting the efficiency.

To run the PIOP in the collaborative setting, we extend the notations of secret sharing to polynomials. We use $[f(Y)] := ([f(\omega_Y^0)], [f(\omega_Y^1)], \dots, [f(\omega_Y^{\ell-1})])$ to denote the vector of (n, t) Shamir's shares of the ℓ evaluations of $f(Y)$. For a degree- (ℓ, T) bi-variate polynomial $g(Y, X)$, we write $[[g(Y, X)]] := ([[g(Y, \omega_X^0)]], [[g(Y, \omega_X^1)]], \dots, [[g(Y, \omega_X^{T-1})]])$, a vector of T elements, where each element $[[g(Y, \omega_X^j)]]$ is an (n, t, ℓ) packed secret sharing of ℓ values $(g(\omega_Y^0, \omega_X^j), g(\omega_Y^1, \omega_X^j), \dots, g(\omega_Y^{\ell-1}, \omega_X^j))$. In other words, we packed secret share the evaluations of the variable Y dimension.

Protocol 1 (Polynomial IOP). The following oracles defined by the circuit are computed by $\mathcal{V}$ or sent by $\mathcal{P}$: $\{Q_{\text{add}}(Y, X), Q_{\text{mult}}(Y, X), Q_{\text{cst}}(Y, X), \sigma_{Y,a}(Y, X), \sigma_{Y,b}(Y, X), \sigma_{Y,o}(Y, X), \sigma_{X,a}(Y, X), \sigma_{X,b}(Y, X), \sigma_{X,o}(Y, X)\}$

1. $\mathcal{P}$ picks blinding values $r_1, r_2, \dots, r_9$ and computes the oracles $A(Y, X) = (r_1X + r_2Y + r_3)V_X(X) + \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y)a_i(X)$, $B(Y, X) = (r_4X + r_5Y + r_6)V_X(X) + \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y)b_i(X)$, $O(Y, X) = (r_7X + r_8Y + r_9)V_X(X) + \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y)o_i(X)$.
2. $\mathcal{V}$ sends random challenges η_Y, η_X, γ to $\mathcal{P}$. $\mathcal{P}$ picks blinding values $r_{10}, \dots, r_{15}$ and computes oracle $Z(Y, X) = (r_{10}X + r_{11}Y + r_{12})XV_Y(Y) + \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y)z_i(X)$ and $Z'(Y, X) = (r_{13}X + r_{14}Y + r_{15})XV_Y(Y) + \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y)z'_i(X)$.
3. $\mathcal{V}$ sends a random challenge λ to $\mathcal{P}$. $\mathcal{P}$ computes $\tilde{H}_X(Y, X)$ as the terms of $G(Y, X) + \lambda P_0(Y, X) + \lambda^2 P_1(Y, X)$ that have any of $r_1, \dots, r_9$ as factors and $\tilde{Q}_X(Y, X) = [(r_{10} - r_{13})X + (r_{11} - r_{14})Y + (r_{12} - r_{15})]XV_Y(Y)$. Then $\mathcal{P}$ picks blinding values $r_{16}, \dots, r_{21}$ and computes oracles $H_X(Y, X) = (r_{16}X + r_{17}Y + r_{18})V_Y(Y) + \tilde{H}_X(Y, X)/V_X(X) + \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y)h_i(X)$ and $Q_X(Y, X) = (r_{19}X + r_{20}Y + r_{21})Y + \tilde{Q}_X(Y, X)/X + \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y)q_i(X)$.
4. $\mathcal{V}$ sends a random evaluation point for $X = \alpha$ to $\mathcal{P}$. $\mathcal{P}$ computes oracles $H_{Y,\alpha}(Y) = \frac{G(Y,\alpha) + \lambda P_0(Y,\alpha) + \lambda^2 P_1(Y,\alpha) - (\alpha^T - 1)H_X(Y,\alpha)}{Y^\ell - 1}$ and $Q_{Y,\alpha}(Y) = \frac{Z(Y,\alpha) - Z'(Y,\alpha) - Q_X(Y,\alpha)\alpha}{Y}$.
5. $\mathcal{V}$ picks a random evaluation point for $Y = \beta$, queries oracles A, B, O, H_X, Z, Z', Q_X at $X = \alpha, Y = \beta$, and oracles $H_{Y,\alpha}, Q_{Y,\alpha}$ at $Y = \beta$. $\mathcal{V}$ also queries oracles defined by the circuit at $X = \alpha, Y = \beta$. $\mathcal{V}$ checks that Equation 5 and 6 holds at $X = \alpha, Y = \beta$, which can be evaluated using the oracle queries. If this equation holds, then $\mathcal{V}$ outputs 1, otherwise 0.

Figure 1: Our polynomial IOP.

$\dots, g(\omega_Y^{\ell-1}, \omega_X^j))$. In other words, we packed secret share the evaluations of the variable Y dimension.

Operations. In addition to the standard MPC protocols for different operations in Section 2.1, we further define the following operations on secret-shared polynomials.

NTT/INTT. Number Theoretic Transform (NTT) transforms the coefficients of a function into its evaluations, and the Inverse Number Theoretic Transform (INTT) does the inverse. We define the following notations:

- $\text{NTT}(f, T, T')$ transforms a polynomial f of size T into its evaluations over a roots-of-unity group of size T' , where $T' \geq T$.
- $\text{INTT}(f, T')$ transforms the T' evaluations back to the coefficients. Note that for a polynomial f of size $T < T'$, it only has T non-zero coefficients.

Here f may refer to both the coefficients and the evaluations, which should be clear in the context.

On a shared polynomial $[f]$, we define $\text{NTT}([f], T, T')$ and $\text{INTT}([f], T')$ as the NTT/INTT on the underlying plaintext polynomial coefficients and evaluations. When using the Cooley-Tukey algorithm, NTT/INTT only involves additions between secret shares and multiplications between a shared value and a public scalar defined by the root of unity. Therefore, each party can execute the same algorithm on their own shares to compute the sharing of the result of NTT/INTT, without any interactions between each other.

When it comes to bi-variate polynomial g , we use the notation $\text{NTT}_X([g], T, T')$, and $\text{INTT}_X([g], T')$ to represent the NTT/INTT operations on the X variable. Note that a degree- (ℓ, T) polynomial g is defined by $\ell \cdot T$ evaluations. $\text{NTT}_X([g], T, T')$ and $\text{INTT}_X([g], T')$ can be computed as ℓ independent executions, each of size T . In the context of MPC with PSS, note that $[g]$ is encoded as T packed secret shares (of pack size ℓ). $\text{NTT}_X([g], T, T')$ and $\text{INTT}_X([g], T')$ perform NTT/INTT on each secret of the PSS independently, which are SIMD computations with additions and scalar multiplications only. Therefore, they can be computed locally by each party without any interactions. This is a major improvement over zkSaaS [20], which involves NTT/INTT on packed secret-shared univariate polynomials. The computations are not SIMD, and thus incur linear communication and computation on the central party.

Polynomial operations. Given the shares of two univariate polynomials f_0, f_1 of size T_0 and T_1 , the polynomial addition and multiplication of $[f_0]$ and $[f_1]$ can be computed as:

$$[f_0] + [f_1] := \text{NTT}(\text{INTT}([f_0], T_0), T_0, T') \quad (7)$$

$$\oplus \text{NTT}(\text{INTT}([f_1], T_1), T_1, T') \quad (8)$$

$$[f_0] \times [f_1] := \text{NTT}(\text{INTT}([f_0], T_0), T_0, T'') \quad (9)$$

$$\otimes \text{NTT}(\text{INTT}([f_1], T_1), T_1, T'') \quad (10)$$

where $T' = \max(T_0, T_1)$, $T'' = T_0 + T_1$, and $\oplus, \otimes$ denote the element-wise addition and multiplication of two vectors. When the degrees are the same, the polynomial addition can be computed directly on the shares without NTT/INTT.

On packed-shared polynomials, we define

$$[[g_0]] + [[g_1]] = \text{NTT}_X(\text{INTT}_X([g_0], T_0), T_0, T') \oplus \text{NTT}_X(\text{INTT}_X([g_1], T_1), T_1, T') \quad (11)$$

$$[[g_0]] \times [[g_1]] = \text{NTT}_X(\text{INTT}_X([g_0], T_0), T_0, T'') \otimes \text{NTT}_X(\text{INTT}_X([g_1], T_1), T_1, T'') \quad (12)$$

Note that $[[g_0]] \times [[g_1]]$ only computes the multiplication of the X variables, but not Y variables. That is, for $g_0(Y, X) = \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y) g_{0,i}(X)$ and $g_1(Y, X) = \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y) g_{1,i}(X)$, $[[g_0]] \times [[g_1]]$ is the PSS of $g(Y, X) = \sum_{i=0}^{\ell-1} L_\ell^{(i)}(Y) (g_{0,i}(X) \cdot g_{1,i}(X))$. This is exactly the polynomial computation we need in our PIOP.

Protocol 2 (Collaborative ZK for our IOP). The following oracles defined by the circuit are computed by $\mathcal{V}$ or committed by $\mathcal{P}$: $\{Q_{\text{add}}(Y, X), Q_{\text{mult}}(Y, X), Q_{\text{cst}}(Y, X), \sigma_{Y,a}(Y, X), \sigma_{Y,b}(Y, X), \sigma_{Y,o}(Y, X), \sigma_{X,a}(Y, X), \sigma_{X,b}(Y, X), \sigma_{X,o}(Y, X)\}$

1. All $\mathcal{P}$ s commit to $A(Y, X), B(Y, X)$ and $C(Y, X)$
2. Given random challenges η_X, η_Y, γ , all $\mathcal{P}$ s compute $[[Z(Y, X)]], [[Z'(Y, X)]]$. They commit to oracles $Z(Y, X), Z'(Y, X)$.
3. Given random challenge λ , all $\mathcal{P}$ s compute $[[H_X(Y, X)]], [[Q_X(Y, X)]]$. They commit to oracles $H_X(Y, X), Q_X(Y, X)$.
4. Given the evaluation point $X = \alpha$, all $\mathcal{P}$ s compute regular shares of $[G(Y, \alpha)], [P_0(Y, \alpha)], [P_1(Y, \alpha)], [H_X(Y, \alpha)], [Z(Y, \alpha)], [Z'(Y, \alpha)], [Q_X(Y, \alpha)]$. They compute $[H_{Y,\alpha}(Y)], [Q_{Y,\alpha}(Y)]$ and commit to oracles $H_{Y,\alpha}(Y), Q_{Y,\alpha}(Y)$.
5. Given the evaluation point for $Y = \beta$, all $\mathcal{P}$ s open the oracles A, B, O, H_X, Z, Z', Q_X at $X = \alpha, Y = \beta$, and oracles $H_{Y,\alpha}, Q_{Y,\alpha}$ at $Y = \beta$. $\mathcal{V}$ also queries oracles defined by the circuit at $X = \alpha, Y = \beta$. $\mathcal{V}$ checks that Equation 5 and 6 holds at $X = \alpha, Y = \beta$, which can be evaluated using the oracle queries. If this equation holds, then $\mathcal{V}$ outputs 1, otherwise 0.

Figure 2: Our Collaborative zkSNARK

4.1 Our PIOP in MPC

In this section, we show how to efficiently compute the PIOP in Figure 1 in MPC. First, the client evaluates the circuit and secret shares the witness $[[A(Y, X)]], [[B(Y, X)]], [[O(Y, X)]]$ using an (n, t, ℓ) PSS where ℓ is the same as the number of subcircuits in the PIOP, and n is the number of provers in the collaborative zkSNARK. The protocol is presented in Figure 2, with the steps matching the plaintext PIOP in Figure 1. Below, we explain in detail how the provers can compute each step in MPC. The verifier remains exactly the same as in Figure 1. For the public polynomials such as $Q_{\text{add}}, Q_{\text{mul}}, Q_{\text{cst}}$, we assume that they are secret-shared among the provers for the ease of presentation. The shares can be obtained easily with access to the circuit, and additional optimizations can be applied to multiplications between private and public polynomials.

Step 1. The provers commit to the oracles $A(Y, X), B(Y, X), O(Y, X)$ using the shares $[[A(Y, X)]], [[B(Y, X)]], [[O(Y, X)]]$ and the collaborative PCS scheme that will be presented in the next section. Note that in order to achieve zero-knowledge, the provers need to add blinding values $r_1, \dots, r_6$ as described in Figure 1. They can be sampled using π_{rand} as PSS and added to the PSS of polynomials directly. The same approach is used on Z, Z', H_X, Q_X . Therefore, we focus on computing the shared polynomials without the blinding values below.

Step 2. For the gate constraint, the provers compute

$$\begin{aligned} [[G(Y, X)]] &= [[Q_{add}(Y, X)]] \times ([[A(Y, X)]] + [[B(Y, X)]] \\ &\quad + [[Q_{mul}(Y, X)]] \times [[A(Y, X)]] \times [[B(Y, X)]] \\ &\quad - [[O(Y, X)]] + [[Q_{cst}(Y, X)]] \end{aligned}$$

using the polynomial additions and multiplications defined above.

Step 3. For the permutation constraint, given random challenges η_X, η_Y, γ from the verifier, the provers first compute

$$\begin{aligned} [[TA(Y, X)]] &= [[A(Y, X)]] + \eta_Y [[\sigma_{Y,a}(Y, X)]] + \eta_X [[\sigma_{X,a}(Y, X)]] + \gamma \\ [[TA'(Y, X)]] &= [[A(Y, X)]] + \eta_Y Y + \eta_X k_a X + \gamma \\ [[TB(Y, X)]] &= [[B(Y, X)]] + \eta_Y [[\sigma_{Y,b}(Y, X)]] + \eta_X [[\sigma_{X,b}(Y, X)]] + \gamma \\ [[TB'(Y, X)]] &= [[B(Y, X)]] + \eta_Y Y + \eta_X k_b X + \gamma \\ [[TO(Y, X)]] &= [[O(Y, X)]] + \eta_Y [[\sigma_{Y,o}(Y, X)]] + \eta_X [[\sigma_{X,o}(Y, X)]] + \gamma \\ [[TO'(Y, X)]] &= [[O(Y, X)]] + \eta_Y Y + \eta_X k_o X + \gamma \end{aligned}$$

Using π_{invert} , the provers compute the inverse the evaluations of $TA, TA', TB, TB', TO, TO'$ at Ω_X using the PSS, and interpret the result as a new bi-variate polynomials of degree (ℓ, T) :

$$\begin{aligned} [[Z(Y, X)]] &= [[TA(Y, X)]]^{-1} + [[TB(Y, X)]]^{-1} + [[TO(Y, X)]]^{-1} \\ [[Z'(Y, X)]] &= [[TA'(Y, X)]]^{-1} + [[TB'(Y, X)]]^{-1} + [[TO'(Y, X)]]^{-1} \end{aligned}$$

To compute P_0, P_1 defined by Equation 3, the prover computes

$$\begin{aligned} [[P_0(Y, X)]] &= \\ & [[TA(Y, X)]] \times [[TB(Y, X)]] \times [[TO(Y, X)]] \times [[Z(Y, X)]] \\ & - [[TA(Y, X)]] \times [[TB(Y, X)]] - [[TA(Y, X)]] \times [[TO(Y, X)]] \\ & - [[TB(Y, X)]] \times [[TO(Y, X)]] \\ [[P_1(Y, X)]] &= \\ & [[TA'(Y, X)]] \times [[TB'(Y, X)]] \times [[TO'(Y, X)]] \times [[Z'(Y, X)]] \\ & - [[TA'(Y, X)]] \times [[TB'(Y, X)]] - [[TA'(Y, X)]] \times [[TO'(Y, X)]] \\ & - [[TB'(Y, X)]] \times [[TO'(Y, X)]] \end{aligned}$$

Given the randomness λ from the verifier, the provers compute

$$\begin{aligned} [[H_X(Y, X)]] &= \frac{[[G(Y, X)]] + \lambda [[P_0(Y, X)]] + \lambda^2 [[P_1(Y, X)]]}{V_X(X)} \\ [[Q_X(Y, X)]] &= \frac{[[Z(Y, X)]] - [[Z'(Y, X)]] - [[W(Y)]]}{X} \end{aligned}$$

where $[[W(Y)]] = [[Z(Y, 0)]] - [[Z'(Y, 0)]]$.

Step 4. Given the evaluation point $X = \alpha$ from the verifier, the provers cannot compute the shares of $H_{Y,\alpha}(Y)$ directly. Taking $G(Y, \alpha)$ in the numerator as an example, by the definition of $G(Y, X)$, the degree in Y is 3ℓ , but the provers only have the PSS $[[G(Y, X)]]$ packing ℓ evaluations at ω_i^j for $i = [\ell]$. Therefore, the provers need to compute from the shares of A, B, O, Z, Z' with degree ℓ in Y . We present the algorithm to compute $[H_{Y,\alpha}(Y)]$:

1. $[[A(Y, \alpha)]] = \sum_{j=0}^{T-1} [[A(Y, \omega_X^j)]] \cdot L_j(\alpha)$
2. $\pi_{\text{ps2rs}}([[A(Y, \alpha)]]) \rightarrow [A(Y, \alpha)]$. Note that $[A(Y, \alpha)]$ is a vector of (n, t) regular shares of ℓ evaluations of $A(Y, \alpha)$.
3. Increase the size of $[A(Y, \alpha)]$ to 4ℓ by NTT(INTT($[A(Y, \alpha), \ell], 4\ell$)).
4. Repeat steps (1)–(3) for $B(Y, X), O(Y, X)$.
5. Compute $[G(Y, \alpha)]$ by Equation 1. Note that this is the shares of 4ℓ evaluations of $G(Y, \alpha)$, which is enough to represent the polynomial and compute $H_{Y,\alpha}(Y)$.
6. Repeat the algorithm to compute $[P_0(Y, \alpha)], [P_1(Y, \alpha)], [H_X(Y, \alpha)]$
7. The provers compute $[H_{Y,\alpha}(Y)] = \frac{[G(Y, \alpha)] + \lambda [P_0(Y, \alpha)] + \lambda^2 [P_1(Y, \alpha)] - (\alpha^T - 1) [H_X(Y, \alpha)]}{V_Y(Y)}$

The provers also compute

$$[Q_{Y,\alpha}(Y)] = \frac{[Z(Y, \alpha)] - [Z'(Y, \alpha)] - [Q_X(Y, \alpha)] \alpha}{Y}$$

As the degree of Y in Z, Z' and Q_X is less than ℓ , the provers can compute $[[Q_{Y,\alpha}(Y)]]$ directly and convert it to $[Q_{Y,\alpha}(Y)]$.

Step 5. Finally, give the evaluation point $Y = \beta$ from the verifier, the provers compute and reconstruct $A(\alpha, \beta), B(\alpha, \beta), O(\alpha, \beta), Z(\alpha, \beta), Z'(\alpha, \beta), H_X(\alpha, \beta), Q(\alpha, \beta)$ and $H_{Y,\alpha}(\beta), Q_{Y,\alpha}(\beta)$. They also open the PCS at these points, which will be presented in the next section.

Complexity analysis. In Steps 1-3, there is a constant number of polynomial multiplications on PSS defined at the beginning of Section 4. Using a PSS scheme with $t = O(n)$ and $\ell = O(n)$, only the element-wise multiplications incur the communication and computation complexity of $O(T)$ per party. NTT_x and INTT_x are non-interactive, and have the computation complexity of $O(T \log T)$. There is also one inverse, which incurs 1 round of interaction with $O(T)$ computation and communication per party. In Step 4, after the evaluations of the polynomials in PSS, the communication and computation from the regular shares are $O(n)$ and $O(\ell \log \ell)$, independent of T . The same happens in Step 5. Therefore, overall, the total computation complexity of our collaborative zkSNARK for the IOP part is $O(T \log T + \ell \log \ell)$, and the communication complexity is $O(T + n)$. In practice, usually $n \ll T$, and thus the prover time of our scheme is $O(\frac{C}{n} \log \frac{C}{n})$.

4.2 Bivariate Polynomial Commitment

In this section, we show how to compile our PIOP to a zk-SNARK in the collaborative setting.

Protocol 3 (Bivariate KZG Polynomial Commitment).

- $\text{pp} \leftarrow \text{PC.KeyGen}(T, \ell)$.
 1. Sample a bilinear map $\text{bp} \leftarrow \text{BilGen}(1^\lambda)$. Sample secrets $s_x, s_y \in \mathbb{F}_p$.
 2. Compute $\tau_{xy}^{(ij)} \leftarrow \{g_1^{L_T^{(i)}(s_x)L_\ell^{(j)}(s_y)}\}$ for all $i \in [T], j \in [\ell]$.
 3. Compute $\tau_y^{(j)} \leftarrow \{g_1^{L_\ell^{(j)}(s_y)}\}$ for $j \in [\ell]$.
 4. Compute $\tau_{x,2} \leftarrow g_2^{s_x}, \tau_{y,2} \leftarrow g_2^{s_y}$.
 5. $\text{pp} = (\text{bp}, \tau_{xy}, \tau_y, \tau_{x,2}, \tau_{y,2})$.
- $c \leftarrow \text{PC.Commit}(f, \text{pp})$.
 1. Compute $c \leftarrow \prod_{i \in [T], j \in [\ell]} (\tau_{xy}^{(ij)})^{f(\omega_x^i, \omega_y^j)}$.
- $(v, \pi) \leftarrow \text{PC.Open}(\alpha, \beta, f, \text{pp})$
 1. Evaluate $v = f(\alpha, \beta)$
 2. Compute $q_x(X, Y), q_y(Y)$ s.t.

$$f(X, Y) - f(\alpha, \beta) = q_x(X, Y)(X - \alpha) + q_y(Y)(Y - \beta)$$
 3. $\pi_1 = \prod_{i \in [T], j \in [\ell]} (\tau_{xy}^{(ij)})^{q_x(\omega_x^i, \omega_y^j)}, \pi_2 = \prod_{j \in [\ell]} (\tau_y^{(j)})^{q_y(\omega_y^j)}$.
 4. Return $(v, (\pi_1, \pi_2))$.
- $\{0, 1\} \leftarrow \text{PC.Verify}(\alpha, \beta, v, c, \pi, \text{pp})$
 1. If $e(c \cdot g_1^{-v}, g_2) = e(\pi_1, g_2^{s_x - \alpha}) \cdot e(\pi_2, g_2^{s_y - \beta})$, output 1; otherwise, output 0.

Figure 3: Bivariate KZG polynomial commitment.

4.2.1 Plain Bivariate KZG Polynomial Commitment

We start with the description of the standard bivariate KZG polynomial commitment scheme [29, 36] in Figure 3. The family of polynomials contains all bivariate polynomials $f(X, Y)$ with degree $T - 1$ in X and $\ell - 1$ in Y . We assume the bivariate polynomial is represented by its evaluations $f(\omega_x^i, \omega_y^j)$ for $i \in [T], j \in [\ell]$.

In our protocol, we also use the standard KZG univariate polynomial commitment in variable Y . The protocol is a special case of the bivariate one by setting $T = 0$, and we omit its description in a separate protocol.

Moreover, in our PIOP in Figure 1, the degree of H_X is $3T - 1$ in X and H_Y is $3\ell - 1$ in Y , as P_0, P_1 are defined as the product of 4 polynomials in Equation 3. We can divide them into 3 separate polynomials of degree $T - 1$ and $\ell - 1$ as in the original Plonk [17]. Alternatively, we can increase pp to include $\tau_{xy}^{(ij)}$ up to $i \in [3T], j \in [\ell]$ and τ_y^j up to $j \in [3\ell]$, and add degree checks for other polynomials of degree less than (T, ℓ) . For example, to check that $c = g_1^{f(s_x, s_y)}$ is a commitment of a polynomial with degree less than (T, ℓ) with pp up to (T', ℓ) for $T' > T$, the prover sends $c' = g^{f(s_x, s_y)s_x^{T'-T}}$ and the verifier

checks that $e(c', g_2) = e(c, g_2^{s_x^{T'-T}})$. Multiple degree bounds can be combined into a single check using the technique in [10]. In the rest of the section, we use (T, ℓ) as the degree of the PCS for simplicity, but all the protocols work for higher degrees used in our PIOP.

4.2.2 Bivariate Polynomial Commitment in MPC

In our setting of MPC, the evaluations of the polynomial f are given by the packed secret sharing $[[f(X, Y)]] := ([[f(\omega_x^0, Y)]]], [[f(\omega_x^1, Y)]]], \dots, [[f(\omega_x^{T-1}, Y)]]]$, a vector of T elements, where each element $[[f(\omega_x^i, Y)]]$ is an (n, t, ℓ) PSS of ℓ values $(f(\omega_x^i, \omega_y^0), g(\omega_x^i, \omega_y^1), \dots, g(\omega_x^i, \omega_y^{\ell-1}))$. To compute the i -th PSS using the Shamir's secret sharing, a random degree- $(t + \ell)$ polynomial h_i is selected such that $h_i(a_j) = f(\omega_x^i, \omega_y^j)$ for pre-selected values a_j for $j \in [\ell]$. Then the share of party $k \in [n]$ is computed as $h_i(b_k)$, where b_k s are again pre-selected values for PSS. For the efficiency of polynomial interpolations, a, b are usually defined by the roots-of-unity, but we use a, b to separate them from ω_x, ω_y in the equations. In this way, the secret value can be computed using the shares as: $f(\omega_x^i, \omega_y^j) = h_i(a_j) = \sum_{k=0}^{n-1} L_n^{(k)}(a_j) h_i(b_k)$, where $L_n^{(k)}$ is the k -th Lagrange polynomial defined by the n -th root-of-unity.

Polynomial commitment with PSS. With the definitions above, the commitment of the bivariate polynomial $f(X, Y)$ is:

$$\begin{aligned} c &= \prod_{i \in [T], j \in [\ell]} (\tau_{xy}^{(ij)})^{f(\omega_x^i, \omega_y^j)} \\ &= \prod_{i \in [T]} \prod_{j \in [\ell]} (\tau_{xy}^{(ij)})^{h_i(a_j)} \\ &= \prod_{i \in [T]} \prod_{j \in [\ell]} (\tau_{xy}^{(ij)})^{\sum_{k=0}^{n-1} L_n^{(k)}(a_j) h_i(b_k)} \end{aligned}$$

By reordering the product and the summation, we have:

$$c = \prod_{k \in [n]} \prod_{i \in [T]} \prod_{j \in [\ell]} (\tau_{xy}^{(ij)})^{L_n^{(k)}(a_j) h_i(b_k)}$$

Then $c = \prod_{k \in [n]} c_k$, where

$$c_k = \prod_{i \in [T]} \left(\prod_{j \in [\ell]} (\tau_{xy}^{(ij)})^{L_n^{(k)}(a_j)} \right)^{h_i(b_k)}. \quad (13)$$

Now each c_k can be computed by party k using her share of PSS individually. This is because $\tau_{xy}, L_n^{(k)}, a, b$ are all public, and $h_i(b_k)$ is the k -th share of the i -th PSS held by party k .

As a result, given $p_{i,k} := \prod_{j \in [\ell]} (\tau_{xy}^{(ij)})^{L_n^{(k)}(a_j)}$, c_k can be computed as an MSM of size T on each party non-interactively. This protocol is summarized in Figure 4. The only difference from the description above is that in order to preserve zero-knowledge, the parties generate a random regular sharing of $[0]$ of threshold t . Then each party k computes c'_k as follows:

$$c'_k = g_1^{L_n^{(k)}(0) \cdot [0]_k} \cdot \prod_{i \in [T]} p_{i,k}^{h_i(b_k)}$$

and publishes c'_k .

To open the polynomial at evaluation point α, β , the provers can compute $[[q_x(X, Y)]] = \frac{[[f(X, Y)] - [[f(\alpha, Y)]]}{X - \alpha}$, where $f(\alpha, \omega_y^j) = \sum_{i \in [T]} L_T^{(i)}(\alpha) f(\omega_x^i, \omega_y^j)$ for $j \in [\ell]$, and thus $[[f(\alpha, Y)]]$ can be computed locally on PSS by the inner product between $[[f(X, Y)]]$ and $(L_T^{(i)}(\alpha))_{i=0}^{T-1}$ without any interaction. Given the PSS of $q_x(X, Y)$, the prover can use the same protocol above to compute $\pi_1 = g_1^{q_x(s_x, s_y)}$. The provers then switch to the regular shares of $[f(\alpha, Y)]$ using π_{ps2rs} . They reconstruct $f(\alpha, \beta)$ after the inner product between $[f(\alpha, Y)]$ and $(L_\ell^{(j)}(\beta))_{j=0}^{\ell-1}$, and compute $[q_y(Y)] = \frac{[f(\alpha, Y)] - [f(\alpha, \beta)]}{Y - \beta}$. Finally, $\pi_2 = g_1^{q_y(s_y)}$ is computed from $[q_y(Y)]$ using a similar protocol on a univariate polynomial. The full protocol of collaborative bivariate polynomial commitment scheme is presented in Figure 5.

The protocol $\pi_{\text{eval-open}}$ (fig. 4) computes the evaluation of a secret-shared polynomial f at a given point and opens the result. The corresponding ideal functionality, $f_{\text{eval-open}}$, is given in section 2.1, and theorem A.2 proves that $\pi_{\text{eval-open}}$ securely computes $f_{\text{eval-open}}$ against semi-honest adversaries.

Complexity analysis. The major advantage of our collaborative bivariate PCS scheme is that given the public parameters and the shares of polynomials, the commitment and the proofs are computed non-interactively at each party using MSMs of size $O(T)$ and $O(\ell)$, and the algorithms are exactly the same as the KZG polynomial commitment on plaintext. Optimizations such as the Pippenger’s algorithm can be applied directly without any change. In PC.Open, to compute $[f(\alpha, Y)]$ and $f(\alpha, \beta)$, a share switching protocol between PSS and regular sharing is executed. However, this was already done in our PIOP (Step 4 of Figure 2), and thus does not introduce any additional overhead when combined with the PIOP protocol. In sum, the computational complexity per party is $O(T)$ group operations (assuming $\ell < T$) and the round complexity is $O(1)$. In PC.KeyGen, with access to s_x, s_y , all $p_{i,k}$ s can be computed in time $O(T \cdot n) = O(C)$. $\tau_y^{(j,k)}$ s can be computed in time $O(\ell \cdot n) < O(T \cdot n)$. Each party only needs to store the public parameters of size $O(T + \ell)$.

4.3 Security Proofs

Theorem 4.1. *The scheme presented in Section 4 is a collaborative zkSNARK by definition 2.1.*

Proof Sketch. The completeness is straight-forward. The knowledge soundness follows from the soundness of the bivariate PIOP as shown in [31]. We only change the permutation constraints to the approach of logarithmic derivatives, and the soundness relies on the proof in [26] and the generalized univariate sumcheck in Lemma 1. The PIOP is compiled to a SNARK using the bivariate KZG PCS, as formalized in [7]. The scheme can be made non-interactive using the Fiat-Shamir transformation [15]. Note that by the definition

Protocol 4 $\pi_{\text{eval-open}}(n, t, \ell, \tau_{xy})$

Input: $[[f(X, Y)]] := ([[\mathbf{f}_0]], \dots, [[\mathbf{f}_{T-1}]])$, where for each $i \in [T]$, $\mathbf{f}_i = (f(\omega_x^i, \omega_y^1), \dots, f(\omega_x^i, \omega_y^{\ell-1}))$.

Pre-processed inputs: each party k holds

$$p_{i,k} := \prod_{j \in [\ell]} (\tau_{xy}^{(ij)})^{L_n^{(k)}(a_j)}$$

for each $i \in [T]$.

1. The parties compute $[0] = \text{share}(\mathbb{F}, 0, t + \ell)$.
2. Each party k computes

$$c'_k = g_1^{L_N^{(k)}(0) \cdot [0]_k} \cdot \prod_{i \in [T]} p_{i,k}^{[[\mathbf{f}_i]]_k}$$

and sends c'_k to party P_1 .

3. P_1 computes $c = \prod_{k \in [n]} c'_k$ and publishes c .

Output: $c = g_1^{f(s_x, s_y)}$ is public.

Figure 4: $\pi_{\text{eval-open}}$.

of knowledge soundness, it can be argued using the plain PIOP and PCS, and the MPC protocols realizing them are not relevant. We omit the formal proof and focus on the t -zero-knowledge in the collaborative setting.

t -Zero-Knowledge. The MPC is computed securely, so the only information revealed to the adversary comes from the secret-shared values that are opened. These values constitute the proof π . Next, the blinding scalars hide the witness and ensure that π satisfies zero-knowledge.

Next, we will explain the proof in more detail.

The MPC is secure. The MPC protocols in section A securely compute the ideal functionalities listed in section 2.1. This is proven in [19] and theorem A.1. Then it suffices to prove the security of our collaborative zkSNARK protocol in a hybrid model with ideal access to the functionalities $f_{\text{rand}}, f_{\text{double-rand}}, f_{\text{pack-mult}}, f_{\text{permute}}, f_{\text{ps2rs}}, f_{\text{invert}}, f_{\text{eval-open}}$. This follows from the modular composition theorem of [8].

Next, the probability that f_{invert} outputs $\perp$ on any invocation is negligible. The inputs to f_{invert} will be non-zero with overwhelming probability over γ . Given that f_{invert} does not output $\perp$, then f_{invert} reveals no information to the adversary about the inputs, except that they are non-zero.

By the perfect secrecy of the secret sharing schemes, the adversary’s view of the collaborative zkSNARK can be simulated given only the values reconstructed by **open**, **popen**, or $f_{\text{eval-open}}$. In the hybrid model, the only values that are reconstructed are the commitments, evaluations, and opening proofs of various polynomials, and these values constitute the zkSNARK proof. It suffices to prove that the zkSNARK proof can be simulated without knowledge of the witness.

The proof can be computed from 21 polynomial evaluations. We will show that the zkSNARK proof can

Protocol 5 (Collaborative Bivariate Polynomial Commitment)

- $pp \leftarrow \text{PC.KeyGen}(T, \ell, n)$.
 1. Sample a bilinear map $bp \leftarrow \text{BilGen}(1^\lambda)$. Sample secrets $s_x, s_y \in \mathbb{F}_p$.
 2. Compute $p_{i,k} := \prod_{j \in [\ell]} (g_1^{L_T^{(j)}(s_x) L_\ell^{(j)}(s_y)})^{L_n^{(k)}(a_j)}$ for all $i \in [T], k \in [n]$.
 3. Compute $\tau_y^{(j,k)} := g_1^{L_n^{(k)}(0) L_\ell^{(j)}(s_y)}$ for $j \in [\ell], k \in [n]$.
 4. Compute $\tau_{x,2} \leftarrow g_2^{s_x}, \tau_{y,2} \leftarrow g_2^{s_y}$.
 5. $pp = (bp, p, \tau_y, \tau_{x,2}, \tau_{y,2})$.
- $c \leftarrow \text{PC.Commit}([f], pp)$.
 1. Generate the regular sharing of $[0]$.
 2. Compute $c'_k = g_1^{L_n^{(k)}(0) \cdot [0]_k} \cdot \prod_{i \in [T]} p_{i,k}^{[f(\omega_x^i, Y)]_k}$, where $[f(\omega_x^i, Y)]_k$ is the share of $f(\omega_x^i, Y)$ at party k .
 3. $c = \prod_{k=0}^{n-1} c'_k$.
- $(v, \pi) \leftarrow \text{PC.Open}(\alpha, \beta, [f], pp)$
 1. Evaluate $[q_x(X, Y)] = \frac{[f(X, Y)] - [f(\alpha, Y)]}{X - \alpha}$. Compute $\pi_1 = \text{PC.Commit}([q_x], pp)$ using the same protocol above.
 2. Compute $[f(\alpha, Y)] = \pi_{\text{ps2rs}}([f(\alpha, Y)])$. Evaluate $[f(\alpha, \beta)]$ and compute $v = f(\alpha, \beta) = \text{open}([f(\alpha, \beta)])$. Compute $[q_y(Y)] = \frac{[f(\alpha, Y)] - [f(\alpha, \beta)]}{Y - \beta}$.
 3. Generate the regular sharing of $[0]$. Compute $\pi_{2,k} = g_1^{L_n^{(k)}(0) \cdot [0]_k} \cdot \prod_{j \in [\ell]} \tau_y^{(j,k) [q_y(\omega_x^j)]_k}$.
 4. $\pi_2 = \prod_{k \in [n]} \pi_{2,k}$.
 5. Return $(v, (\pi_1, \pi_2))$.
- $\{0, 1\} \leftarrow \text{PC.Verify}(\alpha, \beta, v, c, \pi, pp)$
 1. If $e(c \cdot g_1^{-v}, g_2) = e(\pi_1, g_2^{s_x - \alpha}) \cdot e(\pi_2, g_2^{s_y - \beta})$, output 1; otherwise, output 0.

Figure 5: Collaborative Bivariate Polynomial Commitment.

be computed given the evaluations of 7 polynomials – A, B, O, Z, Z', H_X, Q_X – at 3 inputs: $(s_x, s_y), (\alpha, s_y), (\alpha, \beta)$.

We will assume that the witness used in proof generation is valid. If only one party is supplying the witness, then this assumption is reasonable. An honest party will only supply a valid witness, and a dishonest party that supplies the witness has nothing more to learn about the witness from running the protocol.

First, what information about a polynomial f is revealed by the commitment to f and the opening at a particular input (α, β) ? The commitment is $g_1^{f(s_x, s_y)}$, and the opening reveals

$f(\alpha, \beta), g_1^{q_x(s_x, s_y)}$ and $g_1^{q_y(s_y)}$, where

$$q_y(Y) = \frac{f(\alpha, Y) - f(\alpha, \beta)}{Y - \beta}$$

$$q_x(X, Y) = \frac{f(X, Y) - f(\alpha, \beta) - q_y(Y) \cdot (Y - \beta)}{X - \alpha}$$

$$= \frac{f(X, Y) - f(\alpha, Y)}{X - \alpha}$$

All of these values can be computed given 3 evaluations

$$f(s_x, s_y), f(\alpha, s_y), f(\alpha, \beta)$$

as well as the inputs $(s_x, s_y, \alpha, \beta)$.

In the collaborative zkSNARK, the polynomial commitments can be computed from the following evaluations:

$$A(s_x, s_y), B(s_x, s_y), O(s_x, s_y)$$

$$Z(s_x, s_y), Z'(s_x, s_y), H_X(s_x, s_y), Q_X(s_x, s_y)$$

$$H_Y(\alpha, s_y), Q_Y(\alpha, s_y)$$

The provers also open the polynomials to the following values:

$$A(\alpha, \beta), B(\alpha, \beta), O(\alpha, \beta)$$

$$Z(\alpha, \beta), Z'(\alpha, \beta), H_X(\alpha, \beta), Q_X(\alpha, \beta)$$

$$H_Y(\alpha, \beta), Q_Y(\alpha, \beta)$$

Finally, the opening proofs can be computed from the previous evaluations along with the following ones:

$$A(\alpha, s_y), B(\alpha, s_y), O(\alpha, s_y)$$

$$Z(\alpha, s_y), Z'(\alpha, s_y), H_X(\alpha, s_y), Q_X(\alpha, s_y)$$

Note that $H_Y(\alpha, s_y), H_Y(\alpha, \beta), Q_Y(\alpha, s_y)$, and $Q_Y(\alpha, \beta)$ can be computed from the other evaluations as follows:

$$H_Y(\alpha, s_y) = \frac{G(\alpha, s_y) + \lambda P_0(\alpha, s_y) + \lambda^2 P_1(\alpha, s_y) - (\alpha^T - 1) H_X(\alpha, s_y)}{s_y^\ell - 1}$$

$$H_Y(\alpha, \beta) = \frac{G(\alpha, \beta) + \lambda P_0(\alpha, \beta) + \lambda^2 P_1(\alpha, \beta) - (\alpha^T - 1) H_X(\alpha, \beta)}{\beta^\ell - 1}$$

$$Q_Y(\alpha, s_y) = \frac{Z(\alpha, s_y) - Z'(\alpha, s_y) - Q_X(\alpha, s_y) \alpha}{s_y}$$

$$Q_Y(\alpha, \beta) = \frac{Z(\alpha, \beta) - Z'(\alpha, \beta) - Q_X(\alpha, \beta) \alpha}{\beta}$$

Then the proof can be computed from the evaluations of the polynomials A, B, O, Z, Z', H_X, Q_X at inputs $(s_x, s_y), (\alpha, s_y)$, and (α, β) .

The polynomial evaluations are uniformly random. We will show that these 21 evaluations are uniformly random and independent of the witness due to the randomness of the blinding scalars $(r_1, \dots, r_{21})$.

We add the following blinding terms to the polynomials A, B, O, Z, Z', H_X, Q_X :

$A(X, Y)$ includes $R_A(X, Y) := (r_1X + r_2Y + r_3)V_X(X)$

$B(X, Y)$ includes $R_B(X, Y) := (r_4X + r_5Y + r_6)V_X(X)$

$O(X, Y)$ includes $R_O(X, Y) := (r_7X + r_8Y + r_9)V_X(X)$

$Z(X, Y)$ includes $R_Z(X, Y) := (r_{10}X + r_{11}Y + r_{12}) \cdot X \cdot V_Y(Y)$

$Z'(X, Y)$ includes $R_{Z'}(X, Y) := (r_{13}X + r_{14}Y + r_{15}) \cdot X \cdot V_Y(Y)$

$H_X(X, Y)$ includes $R_{H_X}(X, Y) := (r_{16}X + r_{17}Y + r_{18})V_Y(Y)$

$Q_X(X, Y)$ includes $R_{Q_X}(X, Y) := (r_{19}X + r_{20}Y + r_{21})Y$

Let us assume that $(\alpha, \beta, s_x, s_y)$ are 4 distinct values and none of them equals 0 or a root of unity: $(\omega_x^i)_{i \in [T]}$ or $(\omega_y^j)_{j \in [\ell]}$. This occurs with overwhelming probability. Then the evaluations $R_A(s_x, s_y)$, $R_A(\alpha, s_y)$, and $R_A(\alpha, \beta)$ are uniformly random and independent due to the randomness of (r_1, r_2, r_3) . As a result, $A(s_x, s_y)$, $A(\alpha, s_y)$, and $A(\alpha, \beta)$ are uniformly random and independent due to the randomness of (r_1, r_2, r_3) . Crucially, these evaluations of A are also independent of the witness.

By the same reasoning, we can show that the evaluations of the polynomials A, B, O, Z, Z', H_X, Q_X at inputs (s_x, s_y) , (α, s_y) , (α, β) are uniformly random and independent of the witness, due to the randomness of $(r_1, \dots, r_{21})$.

The ZK Simulator: The zero-knowledge simulator samples random values for the 21 polynomial evaluations defined above and uses them to compute the proof. Then it uses the MPC simulator to simulate the adversary's view in the collaborative zkSNARK protocol.

5 Experiments

We have fully implemented our collaborative zkSNARK in Rust, and we present the experimental results in this section. Our implementation extensively uses the Arkworks library [4], including its implementation of finite field arithmetic, FFT/IFFT algorithms, elliptic curves, bilinear pairing and multi-scalar multiplications (MSM). We instantiate the protocols with the BLS12-381 curve. The size of each field element is 48 bytes, and the size of each $\mathbb{G}_1$ element is 48 bytes as well after compression. We also reuse the implementation of zkSaaS for several basic MPC building blocks in Appendix A. We implement other protocols such as π_{invert} ourselves. For network communication, we use the communicator implemented in zkSaaS, which uses TCP/IP under the hood. Our implementation consists of 2700 lines of Rust code.

5.1 Performance of Our Collaborative zk-SNARK

We first measure the performance of our collaborative zk-SNARK scheme, and compare it to the baseline of the original

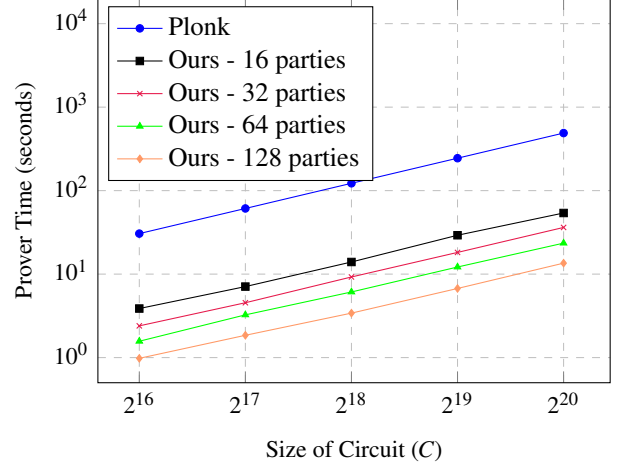


Figure 6: Prover time of our Collaborative zkSNARK

Plonk protocol.

Settings. Two types of machines are used in our experiments. Our protocol was executed on 16–128 AWS instances of c6a.xlarge with 4 vCPUs and 8 GB of memory. Our implementation only utilizes a single core on each machine. The bandwidth between the instances is 10 Gbps. For Plonk, we implemented the protocol ourselves using the same libraries as our scheme. In order to scale Plonk to large circuits, we use a single large AWS instance of c6a-metal with 192 vCPU and 384 GB of memory. The implementation also only utilizes a single thread, which is the same as each party in our collaborative zkSNARK.

We vary the size of the circuit C from 2^{16} to 2^{20} gates, and vary the number of parties n in our scheme from 16 to 128. The batch size of the PSS is always set to $\ell = \frac{n}{4}$. As a result, our scheme can tolerate up to $t = \frac{n}{4} - 1$ parties corrupted by the adversary.

Prover time. Figure 6 shows the prover time of our scheme with different number of parties and Plonk. As shown in the Figure, our collaborative zkSNARK scheme significantly improves the prover time of the zkSNARK, while preserving the privacy of the witness. For $C = 2^{20}$, with 16 parties, the prover time is 54.0 seconds, which is $9\times$ faster than Plonk's prover time of 489.2 seconds. Note that the speedup is more than $\ell = 4$ times as our PIOP breaks the constraint systems into subcircuits with smaller FFTs, and the protocol is simpler due to the use of logarithmic derivatives and univariate sumcheck. With 128 parties, the prover time further decreases to 13.5 seconds, which is $36.2\times$ faster than Plonk.

Micro-benchmark. To further understand the behavior of our scheme, we show the breakdown of the prover time in Table 2 for $C = 2^{20}$. In Plonk as well as most zkSNARKs based on bilinear pairing, the MSM is usually the dominating cost. However, as shown in the table, the PCS only contributes

# of parties	Polynomial IOP	Polynomial Commitment	Total
16	28.4	25.3	53.7
32	23.2	13	36.2
64	16.6	6.9	23.5
128	9.8	3.6	13.4

Table 2: Breakdown of our prover time (seconds) for $C = 2^{20}$.

to 47% of the prover time for $n = 16$ parties in our scheme, and is much faster than the PIOP for $n = 128$. This is due to the compatibility of our collaborative bivariate PCS in Figure 5 with PSS. The computation on each server is almost the same as the plain bivariate KZG PCS on an instance of size $\frac{C}{\ell}$, and the communication overhead is minimal. Therefore, the prover time of the PCS is reduced significantly as the number of parties increases. Meanwhile, the speedup of the PIOP is less due to the communication overhead incurred by MPC protocols (e.g., multiplication, inverse) over PSS.

Proof size. The proof size of our scheme is 960 bytes, regardless of the size of the circuit. This is achieved via the batch opening and the batch degree check of the KZG polynomial commitment. Note that all polynomials in our protocol are opened at the same point, which is simpler to batch than Plonk with two evaluation points.

5.2 Comparison with Prior Work

In this section, we further compare the performance of our collaborative zkSNARK with the prior work of zkSaaS [20] and [33]. Since we were not able to run zkSaaS on multiple machines, we executed the reference implementation in the paper on the large AWS instance of c6a-metal. We simulate one party on each vCPU and set the bandwidth to 10Gbps using the `tc` command. For [33], we executed its reference implementation on the same machine in the “local” mode simulating the running time of the collaborative zkSNARK. As reported in [33, Table 3], the simulated running time is very close to that in the LAN setting.

Figure 7 reports the prover time of 128 parties. As shown in the figure, for a circuit of size 2^{20} , our scheme is $25.9\times$ faster than zkSaaS. This is because zkSaaS requires a central server with $O(C\log C)$ computation and $O(C)$ communication, which is the bottleneck in the setting of our experiment. In fact, its prover time is only slightly faster than plain Plonk in Figure 6 running on the same machine. Similar results were observed with different number of parties from $n = 16$ to $n = 64$. The speedup in [20] was obtained in a setting with a very powerful central server versus a computationally limited client running Plonk. Our scheme does not have such a requirement.

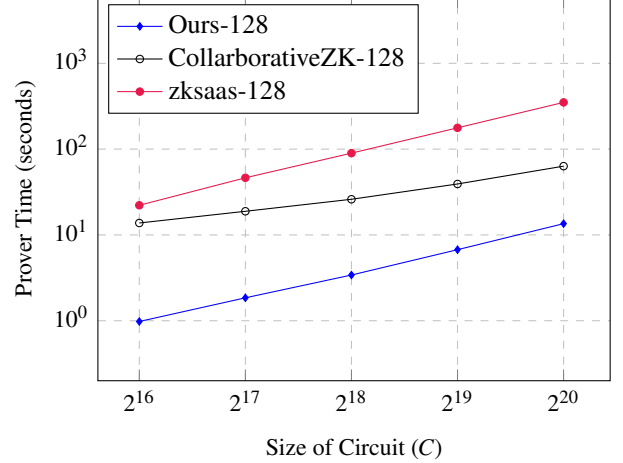


Figure 7: Prover time of our Collaborative zkSNARK

Our prover time is $4.7\times$ faster than the scheme in [33] for $C = 2^{20}$, even though its prover time is linear in the size of the circuit. This is likely due to the fact that the scheme in [33] has $O(\log C)$ rounds, while our scheme has $O(1)$ rounds, as well as the differences of implementation. Moreover, the proof size of [33] is $O(\log C)$, while our proof size is only 960 bytes.

Acknowledgements

This work is supported in part by the AFOSR Award FA9550-24-1-0156, the National Science Foundation (NSF) under Grant No. 2401481, and research grants from the Bakar Fund, J. P. Morgan Faculty Research Award, Supra Inc., Sui Foundation, and the Stellar Development Foundation. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of these institutes.

References

- [1] An Incomplete Guide to Rollups. <https://vitalik.ca/general/2021/01/05/rollup.html>.
- [2] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Liger: Lightweight sublinear arguments without a trusted setup. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2017.
- [3] Diego F Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. Heliopolis: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 302–334. Springer, 2025.
- [4] arkworks contributors. arkworks zksnark ecosystem. <https://arkworks.rs>, 2022.
- [5] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *Proceedings of the Symposium on Security and Privacy (SP)*, 2014, 2014.
- [6] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for r1cs. In *Advances in Cryptology – EUROCRYPT 2019*, pages 103–128, 2019.
- [7] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent snarks from dark compilers. In *Advances in Cryptology – EUROCRYPT 2020*, pages 677–706, 2020.
- [8] Ran Canetti. Security and composition of multiparty cryptographic protocols. *J. Cryptol.*, 13(1):143–202, January 2000.
- [9] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. *Cryptology ePrint Archive*, 2022.
- [10] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zksnarks with universal and updatable srs. In *Advances in Cryptology – EUROCRYPT 2020*, pages 738–768, 2020.
- [11] Alessandro Chiesa, Ryan Lehmkuhl, Pratyush Mishra, and Yinuo Zhang. Eos: Efficient private delegation of {zkSNARK} provers. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6453–6469, 2023.
- [12] Trisha Datta, Binyi Chen, and Dan Boneh. Veritas: Verifying image transformations at scale. *Cryptology ePrint Archive*, 2024.
- [13] Pankaj Dayama, Arpita Patra, Protik Paul, Nitin Singh, and Dhinakaran Vinayagamurthy. How to prove any np statement jointly? efficient distributed-prover zero-knowledge protocols. In *Proceedings on Privacy Enhancing Technologies*, 2022.
- [14] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. *Cryptology ePrint Archive*, 2022.
- [15] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology — CRYPTO’ 86*, 1987.
- [16] Matthew Franklin and Moti Yung. Communication complexity of secure computation. In *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*, pages 699–710, 1992.
- [17] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, 2019.
- [18] Mariana Gama, Emad Heydari Beni, Jiayi Kang, Jannik Spiessens, and Frederik Vercauteren. Blind zksnarks for private proof delegation and verifiable computation over encrypted data. *Cryptology ePrint Archive*, 2024.
- [19] Sanjam Garg, Aarushi Goel, Abhishek Jain, Guru-Vamsi Policharla, and Sruthi Sekar. zkSaaS: Zero-knowledge SNARKs as a service. *Cryptology ePrint Archive*, Paper 2023/905, 2023.
- [20] Sanjam Garg, Aarushi Goel, Abhishek Jain, Guru-Vamsi Policharla, and Sruthi Sekar. {zkSaaS}:{Zero-Knowledge}{SNARKs} as a service. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4427–4444, 2023.
- [21] Sanjam Garg, Aarushi Goel, Abhishek Jain, Bhaskar Roberts, and Sruthi Sekar. Malicious security in collaborative zk-SNARKs: More than meets the eye. *Cryptology ePrint Archive*, Paper 2025/1026, 2025.
- [22] Sanjam Garg, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoodi, Guru-Vamsi Policharla, and Mingyuan Wang. Experimenting with zero-knowledge proofs of training. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1880–1894, 2023.
- [23] Sanjam Garg, Aarushi Goel, and Mingyuan Wang. How to prove statements obliviously? In *Annual International Cryptology Conference*, pages 449–487. Springer, 2024.

- [24] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326, 2016.
- [25] Paul Grubbs, Arasu Arun, Ye Zhang, Joseph Bonneau, and Michael Walfish. {Zero-Knowledge} middleboxes. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 4255–4272, 2022.
- [26] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive*, 2022.
- [27] Yuncong Hu, Pratyush Mishra, Xiao Wang, Jie Xie, Kang Yang, Yu Yu, and Yuwen Zhang. Dfs: Delegation-friendly zk-snark and private delegation of provers. *Cryptology ePrint Archive*, 2025.
- [28] Sanket Kanjalkar, Ye Zhang, Shreyas Gamlur, and Andrew Miller. Publicly auditable mpc-as-a-service with succinct verification and universal setup. In *2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 386–411. IEEE, 2021.
- [29] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology – ASIACRYPT 2010*, pages 177–194.
- [30] Chongrong Li, Pengfei Zhu, Yun Li, Cheng Hong, Wenjie Qu, and Jiaheng Zhang. Hyperpianist: Pianist with linear-time prover and logarithmic communication cost. *Cryptology ePrint Archive*, 2024.
- [31] Tianyi Liu, Tiancheng Xie, Jiaheng Zhang, Dawn Song, and Yupeng Zhang. Pianist: Scalable zk-rollups via fully distributed zero-knowledge proofs. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1777–1793. IEEE, 2024.
- [32] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.
- [33] Xuanming Liu, Zhelei Zhou, Yinghao Wang, Jinye He, Bingsheng Zhang, Xiaohu Yang, and Jiaheng Zhang. Scalable collaborative zk-snark and its application to efficient proof outsourcing. *Cryptology ePrint Archive*, 2024.
- [34] Assa Naveh and Eran Tromer. Photoproof: Cryptographic image authentication for any set of permissible transformations. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 255–271. IEEE, 2016.
- [35] Alex Ozdemir and Dan Boneh. Experimenting with collaborative {zk-SNARKs}:{Zero-Knowledge} proofs for distributed secrets. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 4291–4308, 2022.
- [36] Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. Signatures of correct computation. In *TCC 2013*, pages 222–242, 2013.
- [37] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. pages 238–252, 2013.
- [38] Nicholas Pippenger. On the evaluation of powers and related problems. In *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*, pages 258–263. IEEE Computer Society, 1976.
- [39] Michael Rosenberg, Tushar Mopuri, Hossein Hafezi, Ian Miers, and Pratyush Mishra. Hekaton: Horizontally-scalable zk-snarks via proof aggregation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 929–940, 2024.
- [40] Berry Schoenmakers, Meilof Veeningen, and Niels de Vreede. Trinocchio: Privacy-preserving outsourcing by distributed verifiable computation. In *Applied Cryptography and Network Security: 14th International Conference, ACNS 2016, Guildford, UK, June 19-22, 2016. Proceedings 14*, pages 346–366. Springer, 2016.
- [41] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [42] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. *arXiv preprint arXiv:2404.16109*, 2024.
- [43] Wenhao Wang, Fangyan Shi, Dani Vilardell, and Fan Zhang. Cirrus: Performant and accountable distributed snark. Technical report, Cryptology ePrint Archive, Paper 2024/1873. 2024. url: <https://eprint.iacr.org/2024/1873>.
- [44] Howard Wu, Wenting Zheng, Alessandro Chiesa, Raluca Ada Popa, and Ion Stoica. Dizk: A distributed zero knowledge proof system. In *Proceedings of the 27th USENIX Conference on Security Symposium, SEC’18*, page 675–692, USA, 2018.
- [45] Wenxuan Wu, Soamar Homsy, and Yupeng Zhang. Confidential and verifiable machine learning delegations on the cloud. In *European Symposium on Research in Computer Security*, pages 182–201. Springer, 2024.
- [46] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Advances in Cryptology – CRYPTO*, 2019.

- [47] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh, and Dawn Xiaodong Song. zkbridge: Trustless cross-chain bridges made practical. *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022.
- [48] Yunbo Yang, Yuejia Cheng, Kailun Wang, Xiaoguo Li, Jianfei Sun, Jiachen Shen, Xiaolei Dong, Zhenfu Cao, Guomin Yang, and Robert H Deng. Siniel: Distributed privacy-preserving zkSNARK. *Cryptology ePrint Archive*, 2024.
- [49] Xinxuan Zhang, Ruida Wang, Zeyu Liu, Binwu Xiang, Yi Deng, and Xianhui Lu. Fhe-sNARK vs. SNARK-fhe: From analysis to practical verifiable computation. *Cryptology ePrint Archive*, 2025.

Appendices

A MPC Protocols

We list here several standard MPC protocols based on packed secret sharing. The corresponding ideal functionalities are described in section 2.1. [19] describes the first five protocols – π_{rand} , $\pi_{\text{double-rand}}$, $\pi_{\text{pack-mult}}$, π_{permute} , π_{ps2rs} – and their ideal functionalities, and proves that the protocols securely compute the functionalities against semi-honest adversaries controlling t parties or fewer. For the final protocol π_{invert} , we will prove in theorem A.1 that it securely computes f_{invert} .

Protocol $\pi_{\text{rand}}(n, t, l)$ generate the shares of $n - t$ length- l packed random values across n parties w.r.t. a degree $t + l$ polynomial.

[19] describes the protocol π_{rand} in figure 14, and the ideal functionality in figure 13.

Protocol 1 $\pi_{\text{rand}}(n, t, l)$

Auxiliary Input: Vandermonde matrix $\mathbf{V}_{n, n-t}$ over $\mathbb{F}$.

Private Input: Party i randomly samples $\mathbf{u}_i \leftarrow \mathbb{F}^l$ as their private input.

Steps:

1. Each party P_i calls $[[\mathbf{u}_i]] \leftarrow \text{pshare}(\mathbb{F}, \mathbf{u}_i, t + l)$ to generate the shares of their secret $\mathbf{u}_i$.
2. Each party P_i sends $[[\mathbf{u}_i]]_j$ to P_j .
3. Each party P_i computes

$$([[r_1]]_i, [[r_2]]_i, \dots, [[r_{n-t}]]_i) = \mathbf{V}_{n, n-t}^T ([[u_1]]_i, [[u_2]]_i, \dots, [[u_n]]_i)$$

Output: P_i holds $[[r_1]]_i, [[r_2]]_i, \dots, [[r_{n-t}]]_i$ as shares of some random values: $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_{n-t}$.

Note: The number of shared random vectors is limited to $n - t$ so that adversaries need to corrupt at least t parties to gain some information about the honest parties' secret, matching the security level of Shamir's sharing.

Protocol $\pi_{\text{double-rand}}(n, t, l)$ generates the shares of $n - t$ random vectors of length- l across n parties with regards to a degree $t + l$ polynomial and a degree $n - 1$ polynomial.

[19] describes the protocol $\pi_{\text{double-rand}}$ in figure 16, and the ideal functionality in figure 15.

Protocol 2 $\pi_{\text{double-rand}}(n, t, l)$

Auxiliary Input: Vandermonde matrix $\mathbf{V}_{n, n-t}$ over $\mathbb{F}$.

Private Input: Party i randomly samples $\mathbf{u}_i \leftarrow \mathbb{F}^l$ as their private input.

Steps:

1. Each party P_i calls $[[\mathbf{u}_i]] \leftarrow \text{psshare}(\mathbb{F}, \mathbf{u}_i, t + l)$ and $\langle\langle \mathbf{u}_i \rangle\rangle \leftarrow \text{psshare}(\mathbb{F}, \mathbf{u}_i, n - 1)$ to generate the shares of their secret $\mathbf{u}_i$.
2. Each party P_i sends $[[\mathbf{u}_i]]_j$ and $\langle\langle \mathbf{u}_i \rangle\rangle_j$ to P_j .
3. Each party P_i computes

$$([[r_1]]_i, [[r_2]]_i, \dots, [[r_{n-t}]]_i) = \mathbf{V}_{n, n-t}^T ([[u_1]]_i, [[u_2]]_i, \dots, [[u_n]]_i)$$

$$(\langle\langle r_1 \rangle\rangle_i, \langle\langle r_2 \rangle\rangle_i, \dots, \langle\langle r_{n-t} \rangle\rangle_i) = \mathbf{V}_{n, n-t}^T (\langle\langle u_1 \rangle\rangle_i, \langle\langle u_2 \rangle\rangle_i, \dots, \langle\langle u_n \rangle\rangle_i)$$

Output: P_i holds $[[r_1]]_i, [[r_2]]_i, \dots, [[r_{n-t}]]_i$ and $\langle\langle r_1 \rangle\rangle_i, \langle\langle r_2 \rangle\rangle_i, \dots, \langle\langle r_{n-t} \rangle\rangle_i$ as shares of some random values: $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_{n-t}$.

Protocol $\pi_{\text{pack-mult}}(n, t, l)$ multiplies component-wise the packed shares of two vectors $[[a]]$ and $[[b]]$. The output $[[c]]$ satisfies $\mathbf{c} = \mathbf{a} \odot \mathbf{b}$.

[19] describes the protocol $\pi_{\text{pack-mult}}$ in figure 18, and the ideal functionality in figure 17.

Protocol 3 $\pi_{\text{pack-mult}}(n, t, l)$

Input: Packed sharings $[[\mathbf{a}]]$ and $[[\mathbf{b}]]$.

Steps:

1. The parties invoke $\pi_{\text{double-rand}}(n, t, l)$ to produce $[[\mathbf{r}]]$ and $\langle\langle\mathbf{r}\rangle\rangle$.
2. The parties compute

$$\begin{aligned}\langle\langle\mathbf{c}\rangle\rangle &= [[\mathbf{a}]] \cdot [[\mathbf{b}]] \\ \langle\langle\mathbf{d}\rangle\rangle &= \langle\langle\mathbf{c}\rangle\rangle + \langle\langle\mathbf{r}\rangle\rangle\end{aligned}$$

and send their shares of $\langle\langle\mathbf{d}\rangle\rangle$ to P_1 .

3. P_1 calls $\mathbf{d} \leftarrow \text{popen}(\mathbb{F}, \langle\langle\mathbf{d}\rangle\rangle, n-1)$ to receive $\mathbf{d}$.
4. P_1 calls $[[\mathbf{d}]] \leftarrow \text{pshare}(\mathbb{F}, \mathbf{d}, t+l)$ and sends $[[\mathbf{d}]]_i$ to P_i , for each $i \in [n]$.
5. The parties compute $[[\mathbf{c}]] = [[\mathbf{d}]] - [[\mathbf{r}]]$.

Output: Each P_i holds the share $[[\mathbf{c}]]_i$, where $\mathbf{c} = \mathbf{a} \odot \mathbf{b}$.

Note: This protocol implicitly assumes $2(t+l) < n$ in step 2.

Protocol $\pi_{\text{permute}}(p_{\text{perm}}, n, t, l)$ permutes the entries of a packed-shared vector. It takes as input a packed sharing $[[\mathbf{x}]]$ of l secrets $(x_1, x_2, \dots, x_l)$, and outputs a packed sharing $[[\mathbf{y}]]$ of $(y_1, y_2, \dots, y_l) = p_{\text{perm}}(x_1, x_2, \dots, x_l)$.

π_{permute} and the corresponding ideal functionality are the same as the ones given in [19] in figure 24 and figure 23, respectively, for $k = 1$.

Protocol 4 $\pi_{\text{permute}}(p_{\text{perm}}, n, t, l)$

Input: $[[\mathbf{x}]]$

Pre-processed inputs: $\langle\langle\mathbf{g}\rangle\rangle$ and $[[\mathbf{h}]]$, where $\mathbf{g}$ is a random vector and $\mathbf{h} = p_{\text{perm}}(\mathbf{g})$.

Steps:

1. The parties compute $\langle\langle\mathbf{z}\rangle\rangle = [[\mathbf{x}]] + \langle\langle\mathbf{g}\rangle\rangle$ and send their shares of $\langle\langle\mathbf{z}\rangle\rangle$ to P_1 .
2. P_1 runs $\text{popen}(\mathbb{F}, \langle\langle\mathbf{z}\rangle\rangle, n-1)$ to get $\mathbf{z}$.
3. P_1 computes $\mathbf{z} = p_{\text{perm}}(\mathbf{z})$.
4. P_1 computes $[[\mathbf{z}]] = \text{pshare}(\mathbb{F}, \mathbf{z}, t+l)$ and sends to each P_i their share $[[\mathbf{z}]]_i$.
5. Each P_i computes $[[\mathbf{y}]]_i = [[\mathbf{z}]]_i - [[\mathbf{h}]]_i$.

Output: Each P_i holds $[[\mathbf{y}]]_i$.

Notes: p_{perm} must be known in advance in order to do pre-processing. Also, the pre-processed inputs can be generated by a protocol similar to $\pi_{\text{double-rand}}$.

Protocol $\pi_{\text{ps2rs}}(n, t, l)$ converts a packed sharing $[[\mathbf{x}]]$ to a vector of regular sharings $([x_1], [x_2], \dots, [x_l])$.

[19] describes the protocol π_{ps2rs} in figure 22, and the ideal functionality in figure 21.

Protocol 5 $\pi_{\text{ps2rs}}(n, t, l)$

Input: $[[\mathbf{x}]]$.

Auxiliary Input: Vandermonde matrix $\mathbf{V}_{n,1}$ over $\mathbb{F}$.

Steps:

1. Each party P_i samples $\mathbf{u}_i = (u_{i,1}, u_{i,2}, \dots, u_{i,l}) \leftarrow \mathbb{F}^l$ and computes:

$$\begin{aligned}[[\mathbf{u}_i]] &\leftarrow \text{pshare}(\mathbb{F}, \mathbf{u}_i, t+l) \\ [u_{i,k}] &\leftarrow \text{share}(\mathbb{F}, u_{i,k}, t), \quad \forall k \in [l]\end{aligned}$$

2. For all $i, j \in [n]$, P_i sends $[[\mathbf{u}_i]]_j$ and $[u_{i,1}]_j, \dots, [u_{i,l}]_j$ to P_j .
3. Collectively, the parties compute

$$\begin{aligned}[[\mathbf{r}]] &= \mathbf{V}_{n,1}^T ([[\mathbf{u}_1]], [[\mathbf{u}_2]], \dots, [[\mathbf{u}_n]]) \\ [r_k] &= \mathbf{V}_{n,1}^T ([u_{1,k}], [u_{2,k}], \dots, [u_{n,k}]), \quad \forall k \in [l] \\ [[\mathbf{y}]] &= [[\mathbf{x}]] - [[\mathbf{r}]]\end{aligned}$$

Then each P_i sends their shares of $[[\mathbf{y}]]$ to P_1 .

4. P_1 computes $\mathbf{y} = (y_1, y_2, \dots, y_l) \leftarrow \text{popen}(\mathbb{F}, [[\mathbf{y}]], t+l)$ and sends $\mathbf{y}$ to all parties.
5. The parties collectively compute $[x_k] = y_k + [r_k]$ for each $k \in [l]$.

Output: Each P_i holds $[x_k]_i$ for all $k \in [l]$.

Note: We can convert l regular sharings into 1 packed sharing using a similar protocol to the one above.

Alternatively, if x_k is shared by some polynomial f_k – i.e. $x_k = f_k(b_k)$ for some b_k , and $[x_k]_i = f_k(a_i)$ for some a_i – then party P_i can compute their packed share $[[\mathbf{x}]]_i$ as $\mathbf{f}(a_i) := \sum_{k=1}^l L_{b_k}(a_i) f_k(a_i) = \sum_{k=1}^l L_{b_k}(a_i) [x_k]_i$, where L_{b_k} is the Lagrange polynomial. Note that this assumes b_k are all different for all $k \in [l]$, and a_i remains the same for party i across the k sharings.

Protocol $\pi_{\text{invert}}(n, t, l)$ Takes as input $[[\mathbf{x}]]$ and computes $[[\mathbf{y}]]$, where $y_k = x_k^{-1}$ for each $k \in [l]$.

Protocol 6 $\pi_{\text{invert}}(n, t, l)$

Input: $[[\mathbf{x}]]$ such that $x_k \neq 0$ for all $k \in [l]$.

Steps:

1. The parties call $\pi_{\text{rand}}(n, t, l)$ to obtain $[[\mathbf{r}]]$.
2. The parties compute:

$$\begin{aligned} [[\mathbf{v}]] &= \pi_{\text{pack-mult}}(n, t, l) ([[\mathbf{x}]], [[\mathbf{r}]]) \\ \mathbf{v} &= \text{open}(\mathbb{F}, [[\mathbf{v}]], t + l) \end{aligned}$$

If $v_k = 0$ for some $k \in [l]$, then the parties output $\perp$ and abort the protocol. Otherwise, they continue.

3. The parties compute:

$$\begin{aligned} \mathbf{w} &= (v_1^{-1}, \dots, v_l^{-1}) \\ [[\mathbf{w}]] &= \text{pshare}(\mathbb{F}, \mathbf{w}, t + l) \\ [[\mathbf{y}]] &= \pi_{\text{pack-mult}}(n, t, l) ([[\mathbf{w}]], [[\mathbf{r}]]) \end{aligned}$$

Output: Each P_i holds $[[\mathbf{y}]]_i$ where $y_k = x_k^{-1}$ for each $k \in [l]$.

Theorem A.1. $\pi_{\text{invert}}(n, t, l)$ securely computes $f_{\text{invert}}(n, t, l)$ (defined in section 2.1) against a semi-honest adversary controlling t parties or fewer.

Proof. It suffices to prove security in a hybrid model with ideal access to the functionalities f_{rand} and $f_{\text{pack-mult}}$. This is due to the modular composition theorem of [8] and the fact that π_{rand} and $\pi_{\text{pack-mult}}$ securely compute f_{rand} and $f_{\text{pack-mult}}$ respectively.

First, π_{invert} computes the correct outputs. If $x_k \neq 0$ and $r_k \neq 0$ for all $k \in [l]$, then π_{invert} computes

$$\begin{aligned} \mathbf{v} &= (x_1 \cdot r_1, \dots, x_l \cdot r_l) \\ \mathbf{w} &= ((x_1 \cdot r_1)^{-1}, \dots, (x_l \cdot r_l)^{-1}) \end{aligned}$$

and a random packed sharing of

$$\mathbf{y} = ((x_1 \cdot r_1)^{-1} \cdot r_1, \dots, (x_l \cdot r_l)^{-1} \cdot r_l) = (x_1^{-1}, \dots, x_l^{-1})$$

If instead $x_k = 0$ or $r_k = 0$ for some $k \in [l]$, then $v_k = 0$, so the protocol outputs $\perp$.

Next, the view of the hybrid-model adversary $\mathcal{A}$ can be simulated given only access to f_{invert} . The ideal-world simulator $\mathcal{S}$ runs $\mathcal{A}$ internally and simulates $\mathcal{A}$'s view of the hybrid world. $\mathcal{A}$ specifies the corrupted parties and their shares of $[[\mathbf{y}]]$ and $[[\mathbf{x}]]$, and $\mathcal{S}$ forwards these values to f_{invert} . Then $\mathcal{S}$ only needs to send to $\mathcal{A}$ the values of $\mathbf{v}, \mathbf{w}$ as well as the corrupted parties' shares of $[[\mathbf{w}]]$. Note that $\mathcal{A}$ chooses their shares of $[[\mathbf{r}]], [[\mathbf{v}]], [[\mathbf{y}]]$ by themselves.

$\mathcal{A}$ does not learn anything about $\mathbf{x}$ from $\mathbf{v}$ other than whether each $x_k = 0$. For any $k \in [l]$, if f_{invert} tells $\mathcal{S}$ that $x_k = 0$ or $r_k = 0$, then $\mathcal{S}$ chooses $v_k = 0$. Otherwise, $\mathcal{S}$ samples $v_k \leftarrow \mathbb{F} \setminus \{0\}$. This procedure ensures that $(\mathbf{x}, \mathbf{v})$ will have

the same joint distribution as they do in the hybrid-world protocol. For any value of $x_k \neq 0$, v_k is uniformly distributed over $\mathbb{F} \setminus \{0\}$ over the randomness of r_k .

Next, $\mathcal{S}$ can compute $\mathbf{w}$ deterministically from $\mathbf{v}$. Additionally, $\mathcal{S}$ can sample the corrupted parties' shares of $\mathbf{w}$ uniformly and independently at random because the packed secret-sharing scheme has perfect secrecy for any adversary controlling $\leq t$ parties.

Finally, $\mathcal{S}$ outputs whatever $\mathcal{A}$ outputs at the end of the simulation. $\square$

Theorem A.2. $\pi_{\text{eval-open}}(n, t, l, \tau_{x,y})$ (fig. 4) securely computes $f_{\text{eval-open}}(n, t, l, s_x, s_y)$ (defined in section 2.1) against a semi-honest adversary controlling t parties or fewer.

Proof. First, $\pi_{\text{eval-open}}$ computes the correct output. The output of $\pi_{\text{eval-open}}$ is the following:

$$\begin{aligned} c &= \prod_{k \in [n]} g_1^{L_n^{(k)}(0) \cdot [0]_k} \cdot \prod_{i \in [T]} P_{i,k}^{[[\mathbf{f}_i]]_k} \\ &= g_1^{\sum_{k \in [n]} (L_n^{(k)}(0) \cdot [0]_k + \sum_{i \in [T]} \sum_{j \in [q]} L_T^{(i)}(s_x) L_\ell^{(j)}(s_y) L_n^{(k)}(a_j) \cdot [[\mathbf{f}_i]]_k)} \\ &= g_1^{0 + \sum_{i \in [T]} \sum_{j \in [q]} L_T^{(i)}(s_x) L_\ell^{(j)}(s_y) f(\omega_x^i, \omega_y^j)} = g_1^{f(s_x, s_y)} \end{aligned}$$

Next, we will show that the adversary's view can be simulated with just c .

Each share c'_k is computed as $c'_k = g_1^{e(b_k)}$, where

$$e(b_k) = L_n^{(k)}(0) \cdot h'(b_k) + \sum_{i \in [T]} \sum_{j \in [q]} L_T^{(i)}(s_x) L_\ell^{(j)}(s_y) L_n^{(k)}(a_j) \cdot h_i(b_k)$$

In our notation, h' is the sharing polynomial for $[0]$, and h_i is the sharing polynomial for $[[\mathbf{f}_i]]$. Then the k -th party holds the shares: $[0]_k = h'(b_k)$ and $[[\mathbf{f}_i]]_k = h_i(b_k)$, for all i .

Any $t + \ell + 1$ values in $(c'_k)_{k \in [n]}$ uniquely determine the other c'_k -values. Both $(h_i)_{i \in [T]}$ and h' have degree $\leq t + \ell$, so e has degree $\leq t + \ell$ too. Then $t + \ell + 1$ values from $(c'_k)_k$ uniquely determine e and the remaining c'_k -values.

Furthermore, over the randomness of h' (i.e. $[0]$), e is uniformly random over the set of all polynomials of degree $\leq t + \ell$ such that $\sum_{k \in [n]} e(b_k) = f(s_x, s_y)$.

Next, the adversary's view of $\pi_{\text{eval-open}}$ can be simulated given only $g_1^{f(s_x, s_y)}$. The simulator $\mathcal{S}$ gets as input the adversary's t shares of $[[f(X, Y)]]$ and also receives $g_1^{f(s_x, s_y)}$ from the ideal functionality. Then $\mathcal{S}$ samples t shares of $[0]$ uniformly at random, and samples ℓ of the honest parties' c'_k -values uniformly at random. Then $\mathcal{S}$ computes the c'_k -values of the remaining honest parties to be consistent with the values sampled so far. $\square$